

Vježba 3 - Grananja i petlje u Pythonu

Uvjetno izvršavanje naredbi, ponavljanje i obrada više vrijednosti

AUTHOR

Elvis Starić

U prethodnoj vježbi zapisivali smo podatke u varijable, računali, obrađivali tekst i provjeravali uvjete. Programi su se uglavnom izvršavali redom, od prve prema posljednjoj naredbi. Sada ćemo zapisivati postupke u kojima se pojedine naredbe izvršavaju samo u određenim okolnostima ili se isti blok naredbi ponavlja više puta.

Napravite mapu **Vježba 3** i u nju spremajte primjere i svoja rješenja. Svaki potpuni primjer u skripti može se pokrenuti samostalno. Kod se izvršava na vašem računalu. Očekivani ispis i objašnjenje možete otvoriti u skripti klikom na strelicu.

1 Grananje programa naredbom if

1.1 Uvjetno izvršavanje naredbom if

Programi iz prethodne vježbe uglavnom su izvršavali svaku naredbu redom. Kada Python dođe do poziva funkcije `print()`, poruka se ispisuje neovisno o vrijednostima koje smo prije toga izračunali. Takav redoslijed nije dovoljan kada se neka naredba smije izvršiti samo pod određenim uvjetom.

Zamislite program koji provjerava je li student ostvario najmanje 60 bodova potrebnih za sudjelovanje u radionici. Već znamo usporediti broj bodova s pragom:

```
bodovi = 72
print(bodovi >= 60)
print("Student može sudjelovati u radionici.")
```

Prije pokretanja programa pratite naredbe redom. Prvi `print()` prikazuje rezultat usporedbe, a drugi prikazuje poruku. Postoji li u kodu išta što Pythonu govori da drugu poruku smije ispisati samo kada je usporedba istinita?

▼ Prikaži ispis i objašnjenje problema

Za početnu vrijednost 72 program ispisuje:

```
True
Student može sudjelovati u radionici.
```

Ako vrijednost promijenimo u 45, program ispisuje:

```
False
Student može sudjelovati u radionici.
```

Usporedba `bodovi >= 60` ispravno vraća `False`, ali program nakon toga nastavlja na sljedeći redak. Logička vrijednost sama ne upravlja drugim pozivom funkcije `print()`. Python ne zaključuje da je poruka povezana s prethodnom provjerom samo zato što je zapisana u sljedećem retku.

Potrebna nam je naredba koja mijenja uobičajeni tijek izvršavanja: ako je uvjet zadovoljen, određeni dio programa treba izvršiti, a ako nije, taj dio treba preskočiti. Za to koristimo naredbu `if`.

💡 Grananje i naredba if

Grananje je upravljačka struktura koja, ovisno o rezultatu jednog ili više uvjeta, određuje koji će se blok naredbi izvršiti.

Naredba `if` provjerava uvjet i izvršava pripadajući blok naredbi kada se uvjet tumači kao istinit. Ako je uvjet lažan, taj se blok preskače.

Osnovni zapis u Pythonu izgleda ovako:

```
if uvjet:
    naredba_koja_ovisi_o_uvjetu
```

Svaki dio zapisa ima određenu ulogu:

Dio zapisa	Značenje
<code>if</code>	Ključna riječ kojom započinjemo uvjetno izvršavanje
<code>uvjet</code>	Izraz čiji se rezultat provjerava. U početnim primjerima to je usporedba koja daje <code>True</code> ili <code>False</code>
<code>:</code>	Označava da nakon zaglavlja slijedi blok naredbi
Uvučena naredba	Izvršava se samo kada je uvjet istinit

Riječ `uvjet` i naziv `naredba_koja_ovisi_o_uvjetu` u općem zapisu nisu posebne Pythonove naredbe. Oni predstavljaju mjesta na koja u stvarnom programu upisujemo konkretnu provjeru i naredbu koju želimo izvršiti.

Primijenimo taj zapis na provjeru bodova:

```
bodovi = 72

if bodovi >= 60:
    print("Student može sudjelovati u radionici.")

print("Provjera je završena.")
```

Zaglavlje `if bodovi >= 60:` možemo pročitati kao: „Ako su bodovi veći od ili jednaki `60`, izvrši pripadajući blok.” Izraz `bodovi >= 60` najprije se izračunava. Za vrijednost `72` rezultat je `True`, pa Python ulazi u blok i ispisuje poruku o sudjelovanju.

Posljednja poruka nije dio grananja. Ona se nalazi nakon bloka i izvršava se neovisno o rezultatu uvjeta. Naredba `if` zato **ne završava** izvršavanje programa kada je uvjet lažan, preskače samo naredbe koje joj pripadaju.

Sada ako opet postavimo broj bodova na `45`, a ostatak programa ostavimo jednakim:

```
bodovi = 45

if bodovi >= 60:
    print("Student može sudjelovati u radionici.")

print("Provjera je završena.")
```

Usporedba `45 >= 60` daje `False`, pa Python ne ulazi u uvučeni blok. Poruka `Student može sudjelovati u radionici.` zato se ne ispisuje. Program se nastavlja prvom naredbom nakon bloka:

Provjera je završena.

Uvlačenje i blok naredbi

U prethodnoj vježbi većinu naredbi pisali smo od početka retka. Kod grananja položaj naredbe postaje dio značenja programa. Python pomoću uvlačenja prepoznaje koje naredbe ovise o uvjetu.

Blok naredbi

Blok naredbi skup je naredbi koje pripadaju istoj upravljačkoj strukturi. U Pythonu sve naredbe istog bloka moraju biti jednako uvučene. Povratak na prethodnu razinu uvlačenja označava završetak bloka.

Blok naredbe `if` može sadržavati više redaka. Ako je uvjet istinit, izvršavaju se redom svi redci u tom bloku:

```
broj_prijavljenih = 12

if broj_prijavljenih > 0:
    print("Postoje prijavljeni polaznici.")
    print("Pripremite materijale.")

print("Broj prijavljenih:", broj_prijavljenih)
```

▼ Prikaži ispis i pripadnost naredbi

```
Postoje prijavljeni polaznici.
Pripremite materijale.
Broj prijavljenih: 12
```

Oba uvučena poziva funkcije `print()` pripadaju bloku naredbe `if`. Python ih izvršava jer je usporedba `12 > 0` istinita. Posljednji poziv nije uvučen, pa ne pripada bloku i izvršava se u svakom slučaju.

Ako postavimo `broj_prijavljenih = 0`, preskaču se obje uvučene poruke, a program ispisuje samo:

```
Broj prijavljenih: 0
```

Uobičajeno je svaku novu razinu uvući za četiri razmaka. Za uvlačenje možemo pritisnuti tipku `Tab`. VS Code u Python datoteci njome uobičajeno umeće jednu razinu uvlačenja, odnosno četiri razmaka. VS Code uvlačenje najčešće napravi automatski i nakon dvotočke. Prazan redak možemo koristiti radi čitljivosti, ali on ne određuje završetak bloka. Završetak je vidljiv po tome što se sljedeća naredba vratila na prethodnu razinu uvlačenja.

Uvlačenje određuje kada se naredba izvršava

Nemojte uvlačenje promatrati samo kao vizualno uređivanje koda. Pomicanje naredbe ulijevo ili udesno može promijeniti rezultat programa. Naredbe istog bloka poravnajte jednako i nemojte miješati tabulatore i razmake. Ako nakon zaglavlja `if` nema uvučenog retka, Python će prijaviti pogrešku.

Pogledajmo kako samo jedan položaj naredbe mijenja značenje programa:

```
broj_prijavljenih = 0

if broj_prijavljenih > 0:
    print("Postoje prijavljeni polaznici.")

print("Pripremite materijale.")
```

Program ispisuje **Pripremite materijale.** iako nema prijavljenih polaznika. Kod je sintaksno ispravan, ali druga poruka nije uvučena i zato ne ovisi o uvjetu. Ako se materijali trebaju pripremiti samo kada postoji barem jedan polaznik, i ta naredba mora biti dio bloka.

Pregled novih naredbi

Naredba	Namjena
<code>if uvjet:</code>	Izvršava pripadajući blok kada je uvjet istinit

Zadatak

U spremištu je 7 bilježnica. Ako ih je manje od 10, program treba ispisati poruku da je potrebno naručiti nove bilježnice i izračunati koliko ih nedostaje do zalihe od 20. Na kraju programa uvijek treba ispisati trenutačnu zalihu. Kako biste zapisali taj postupak? Provjerite ga i kada je na zalihi 15 bilježnica.

▼ Prikaži moguće rješenje i ispis

```
zaliha = 7

if zaliha < 10:
    print("Potrebno je naručiti bilježnice.")
    print("Za naručiti:", 20 - zaliha)

print("Trenutačna zaliha:", zaliha)
```

Potrebno je naručiti bilježnice.
Za naručiti: 13
Trenutačna zaliha: 7

Za zalihu 15 izvršava se samo posljednji ispis. Izračun količine za naručivanje pripada istom bloku kao poruka o narudžbi.

2 Odabir između više mogućnosti

2.1 Dvije mogućnosti: if i else

Prethodni program ispisivao je posebnu poruku samo kada je uvjet bio zadovoljen. Često želimo odgovarajuću poruku i u suprotnom slučaju: prijava je prihvaćena ili odbijena, zaliha je dovoljna ili nedovoljna.

Za takav odabir koristimo `else`.

Bez nove naredbe mogli bismo zapisati dvije odvojene provjere:

```
broj_polaznika = 24
broj_mjesta = 20

if broj_polaznika <= broj_mjesta:
    print("Svi polaznici imaju mjesto.")

if broj_polaznika > broj_mjesta:
    print("Nema dovoljno mjesta.")
```

Obje provjere opisuju istu odluku: mjesta ima dovoljno ili ga nema dovoljno. Drugi se slučaj događa upravo kada prvi uvjet nije zadovoljen. Zato nije potrebno zasebno pisati suprotan uvjet `broj_polaznika > broj_mjesta`, dovoljno je navesti što se treba dogoditi u svim preostalim slučajevima. Za to koristimo granu `else`.

Alternativna grana else

Grana `else` sadrži naredbe koje se izvršavaju kada uvjet pripadajuće naredbe `if` nije zadovoljen. U strukturi `if-else` izvršava se točno jedna od dviju grana. Iza `else` ne pišemo novi uvjet, nego samo dvotočku.

Strukturu možemo čitati kao jedno pitanje s dvama mogućim putovima:

```
if uvjet:
    naredbe_za_istinit_uvjet
else:
    naredbe_za_lazan_uvjet

naredbe_nakon_grananja
```

Python najprije izračunava samo uvjet iza `if`. Ako je rezultat istinit, izvršava prvi blok i preskače blok `else`. Ako je rezultat lažan, preskače prvi blok i izvršava `else`. Nakon odabrane grane oba se puta ponovno spajaju te program nastavlja naredbama nakon grananja. Blokovi se ne izvršavaju jedan nakon drugoga.

Naredbe `if` i `else` moraju biti poravnate jer pripadaju istoj strukturi. Njihovi blokovi uvučeni su za jednu razinu. Iza `else` ne zapisujemo uvjet zato što ta grana već znači „u svim preostalim slučajevima”.

```

broj_polaznika = 24
broj_mjesta = 20

if broj_polaznika <= broj_mjesta:
    print("Svi polaznici imaju mjesto.")
else:
    print("Nedostaje mjesta:", broj_polaznika - broj_mjesta)

print("Provjera kapaciteta je završena.")

```

▼ Prikaži ispis i objašnjenje

Nedostaje mjesta: 4
Provjera kapaciteta je završena.

Uvjet `24 <= 20` je lažan, pa se izvršava blok `else`. Za točno 20 polaznika izvršila bi se prva grana jer je jednakost uključena. Naredbe `if` i `else` poravnate su na istoj razini, a sadržaj svake grane uvučen je za jednu razinu.

Za `broj_polaznika = 18` usporedba `18 <= 20` bila bi istinita. Program bi ispisao `Svi polaznici imaju mjesto.`, preskočio izračun nedostajućih mjesta i zatim ispisao završnu poruku.

2.2 Više mogućnosti: if, elif i else

Za neke podatke dvije kategorije nisu dovoljne. Primjerice, temperaturu u prostoriji želimo opisati kao prenisiku, prihvatljivu ili previsoku. U takvom postupku treba provjeriti više uvjeta, ali odabrati samo jednu kategoriju.

Tri kategorije možemo pokušati zapisati samostalnim provjerama:

```

temperatura = 23

if temperatura < 18:
    print("Temperatura je prenisika.")

if temperatura >= 18 and temperatura <= 25:
    print("Temperatura je prihvatljiva.")

if temperatura > 25:
    print("Temperatura je previsoka.")

```

Program radi za zadane granice, ali istu odluku opisuje trima odvojenim uvjetima. Granice se ponavljaju, a iz zapisa nije odmah vidljivo da želimo točno jednu poruku. Treba nam način da nakon neuspjele prve provjere prijedemo na sljedeću mogućnost i zaustavimo odabir čim jednu pronađemo.

💡 Dodatna provjera elif

Grana `elif` uvodi sljedeći uvjet u povezanom grananju. Python ga provjerava samo ako prethodni uvjeti u toj strukturi nisu bili zadovoljeni. Izvršava se prva grana sa zadovoljenim uvjetom. Završni `else`, ako je naveden,

obrađuje slučajeve koji nisu zadovoljili nijedan prethodni uvjet.

Povezano grananje s više mogućnosti ima ovaj opći oblik:

```
if prvi_uvjet:
    naredbe_prve_grane
elif drugi_uvjet:
    naredbe_druge_grane
elif treci_uvjet:
    naredbe_trece_grane
else:
    naredbe_za_preostale_slucajeve
```

Python provjerava uvjete odozgo prema dolje. Čim pronađe prvi istinit uvjet, izvršava njegov blok i preskače cijeli ostatak strukture. Zato kasniji uvjet može biti istinit, ali se uopće neće provjeriti ako je neka ranija grana već odabrana. Završni `else` nema uvjet i izvršava se samo kada su svi prethodni uvjeti bili lažni.

Zadatak: Temperaturu manju od `18` stupnjeva označite kao prenisku, temperaturu od `18` do `25` uključivo kao prihvatljivu, a višu od `25` kao previsoku.

```
temperatura = 23

if temperatura < 18:
    print("Temperatura je preniska.")
elif temperatura <= 25:
    print("Temperatura je prihvatljiva.")
else:
    print("Temperatura je previsoka.")
```

▼ Prikaži ispis i objašnjenje

Temperatura je prihvatljiva.

Prvi uvjet je lažan, pa Python provjerava drugi. Drugi je istinit i njegova se poruka ispisuje. Nakon toga preskače ostatak povezanog grananja.

U grani `elif` nije potrebno ponovno pisati `temperatura >= 18`: do te grane dolazimo tek kada je provjera `temperatura < 18` bila lažna. Za uobičajene brožčane vrijednosti iz ovog primjera to znači da je temperatura barem `18`. Ipak, uvjete uvijek treba tumačiti zajedno s redoslijedom kojim su zapisani.

Tijek za nekoliko karakterističnih vrijednosti izgleda ovako:

Temperatura	<code>temperatura < 18</code>	Provjerava li se <code>temperatura <= 25</code> ?	Odabrana grana
<code>17</code>	<code>True</code>	Ne	Preniska temperatura
<code>18</code>	<code>False</code>	Da, rezultat je <code>True</code>	Prihvatljiva temperatura
<code>25</code>	<code>False</code>	Da, rezultat je <code>True</code>	Prihvatljiva temperatura

Temperatura	Provjerava li se		Odabrana grana
	<code>temperatura < 18</code>	<code>temperatura <= 25 ?</code>	
26	False	Da, rezultat je False	Završni <code>else</code> , previsoka temperatura

Broj grana `elif` nije ograničen na jednu. Završni `else` nije obavezan. Ako ga nema i nijedan uvjet nije zadovoljen, program preskače sve grane i nastavlja nakon strukture. Izostavljanje `else` ima smisla kada za preostale slučajeve doista ne trebamo ništa napraviti. Ako svaki mogući podatak mora pripasti nekoj kategoriji, završna grana pomaže da nijedan slučaj ne ostane bez obrade.

2.3 Redoslijed uvjeta i preklapanje granica

Ako se uvjeti preklapaju, njihov redoslijed određuje rezultat. Za bodove `92` istinite su i usporedba `bodovi >= 60` i usporedba `bodovi >= 90`. Python ne traži „najbolju” ili najstrožu granu, nego izvršava prvu zadovoljenu.

Redoslijed uvjeta određujemo prema tome koje vrijednosti svaka grana treba obuhvatiti. Kod pragova koje provjeravamo operatorom `>=` često krećemo od najvišeg prema nižima, kako viši rezultat ne bi prerano završio u široj kategoriji nižeg praga. Kod granica koje provjeravamo operatorom `<` često krećemo od najniže prema višima. To nije pravilo Pythonove sintakse, nego način da se preklapajući uvjeti poredaju tako da svaka vrijednost završi u namijenjenoj kategoriji.

Sljedeći program je sintaksno ispravan, ali ne prepoznaje izvrsne rezultate kako je zamišljeno:

```
bodovi = 92

if bodovi >= 60:
    print("Zadovoljen prag.")
elif bodovi >= 90:
    print("Izvrstan rezultat.")
else:
    print("Prag nije zadovoljen.")
```

▼ Prikaži problem i ispravak

Program ispisuje `Zadovoljen prag.`. Grana za najmanje `90` bodova nikada ne dolazi na red za takve rezultate jer oni već zadovoljavaju prvi uvjet. Ako želimo ispisati jednu kategoriju, provjere možemo poredati od višeg praga prema nižem:

```
bodovi = 92

if bodovi >= 90:
    print("Izvrstan rezultat.")
elif bodovi >= 60:
    print("Zadovoljen prag.")
else:
    print("Prag nije zadovoljen.")
```

Izvrstan rezultat.

Druga grana sada zapravo znači: bodovi nisu barem 90, ali jesu barem 60. Taj prvi dio ne moramo ponovno zapisati jer je već poznat iz neuspjele prve provjere. Takvo čitanje prethodno isključenih mogućnosti važno je za razumijevanje svake grane `elif`.

2.4 Više samostalnih if naredbi

Ponekad želimo ispisati više poruka jer se provjere odnose na različite stvari. Student istodobno može ostvariti bodovni prag i imati predan projekt. Tada odabir samo jedne poruke ne bi odgovarao zadatku.

```
bodovi = 92
projekt_predan = True

if bodovi >= 60:
    print("Bodovni prag je zadovoljen.")

if projekt_predan:
    print("Projekt je predan.")
```

▼ Prikaži ispis i objašnjenje

Bodovni prag je zadovoljen.
Projekt je predan.

Svaki `if` zasebna je provjera i oba bloka mogu se izvršiti. Budući da `projekt_predan` već sadrži logičku vrijednost, možemo izravno napisati `if projekt_predan:`.

Za razliku od povezanog grananja, završetak prve naredbe `if` nema utjecaja na početak druge. Python nakon prve provjere nastavlja redom i dolazi do nove, neovisne provjere. Zato program može ispisati obje poruke, samo prvu, samo drugu ili nijednu, ovisno o vrijednostima.

⚠ Odaberite strukturu prema značenju provjera

Više samostalnih naredbi `if` može izvršiti više blokova. Povezano grananje `if-elif-else` odabire jednu granu. Te strukture nisu zamjenjive samo zato što sadrže slične uvjete.

🔔 Pregled novih naredbi

Naredba	Namjena
<code>elif uvjet:</code>	Provjerava sljedeću mogućnost ako prethodne nisu odabrane
<code>else:</code>	Izvršava alternativni blok kada nijedan prethodni uvjet u grananju nije zadovoljen

Zadatak

Dostava narudžbe ovisi o vrijednosti kupljenih proizvoda. Za iznos manji od 30 eura dostava stoji 5 eura, za iznos od 30 do manje od 60 eura stoji 2 eura, a od 60 eura nadalje je besplatna. Za kupnju od 45 eura izračunajte dostavu i ukupan iznos za plaćanje.

▼ Prikaži moguće rješenje i ispis

```
iznos = 45

if iznos < 30:
    dostava = 5
elif iznos < 60:
    dostava = 2
else:
    dostava = 0

print(f"Dostava: {dostava:.2f} EUR")
print(f"Ukupno: {iznos + dostava:.2f} EUR")
```

Dostava: 2.00 EUR

Ukupno: 47.00 EUR

Vrijednost `dostava` dodjeljuje se u svakoj grani pa je dostupna nakon grananja.

3 Povezivanje i ugnježdivanje uvjeta

3.1 Složeni uvjeti

Dosad smo najčešće odlučivali prema jednoj usporedbi. U stvarnom pravilu može biti važno više podataka: broj bodova, broj dolazaka i predaja rada. Logičke operatore iz prethodne vježbe sada koristimo kao dio uvjeta naredbe `if`.

Pretpostavimo da su za sudjelovanje istodobno potrebni bodovi i dolasci. Sljedeći pokušaj provjerava ih odvojeno:

```
bodovi = 68
dolasci = 2

if bodovi >= 60:
    print("Uvjet sudjelovanja je zadovoljen.")

if dolasci >= 3:
    print("Uvjet sudjelovanja je zadovoljen.")
```

Program ispisuje da je uvjet zadovoljen već zato što je prolazna samo prva provjera. Dvije samostalne naredbe `if` ne znače da oba uvjeta moraju vrijediti. Potreban nam je jedan uvjet koji povezuje obje provjere.

Za `and` obje provjere moraju biti istinite, za `or` dovoljna je barem jedna, a `not` obrće istinitost uvjeta. Prije pisanja koda korisno je pravilo izreći običnim jezikom i jasno razlikovati „i” od „ili”.

Operator	Kada je cijeli uvjet istinit?	Primjer pravila
<code>and</code>	Kada su istinite obje povezane provjere	Student ima dovoljno bodova i dovoljno dolazaka
<code>or</code>	Kada je istinita barem jedna povezana provjera	Potrebna je provjera zbog premalo dolazaka ili nepredanog rada
<code>not</code>	Kada je provjera iza operatora lažna	<code>not rad_predan</code> znači da rad nije predan

Operator ne mijenja same podatke. On povezuje ili obrće rezultate provjera i stvara novu logičku vrijednost koju `if` zatim koristi za odabir bloka.

```
bodovi = 68
dolasci = 4
rad_predan = False

uvjet_sudjelovanja = bodovi >= 60 and dolasci >= 3 and rad_predan

if uvjet_sudjelovanja:
    print("Svi uvjeti sudjelovanja su zadovoljeni.")
else:
    print("Uvjet sudjelovanja nije zadovoljen.")
```

▼ Prikaži ispis i objašnjenje

Uvjet sudjelovanja nije zadovoljen.

Operator `and` povezuje tri provjere. Student ima dovoljno bodova i dolazaka, ali vrijednost `rad_predan` iznosi `False`. Budući da sve tri provjere moraju biti istinite, cijeli izraz daje `False` i izvršava se grana `else`.

Pojedinačne provjere možemo spremiti u zasebne varijable kako bismo lakše pratili rezultat:

```
dovoljno_bodova = bodovi >= 60
dovoljno_dolazaka = dolasci >= 3
uvjet_sudjelovanja = dovoljno_bodova and dovoljno_dolazaka and rad_predan
```

Vrijednosti `dovoljno_bodova`, `dovoljno_dolazaka` i `rad_predan` redom su `True`, `True` i `False`, pa njihov rezultat s operatorom `and` iznosi `False`. Takve pomoćne varijable nisu obavezne, ali mogu pomoći pri učenju i provjeri složenijeg izraza.

Isti razlog za odbijanje možemo promatrati iz suprotnog smjera. Dodatna provjera potrebna je ako student nema dovoljno bodova, nema dovoljno dolazaka **ili** nije predao rad:

```
potrebna_provjera = bodovi < 60 or dolasci < 3 or not rad_predan
```

U ovom slučaju prve dvije provjere daju `False`, a `not rad_predan` daje `True`. Kod operatora `or` dovoljna je jedna istinita provjera, pa `potrebna_provjera` iznosi `True`. Oba izraza opisuju isto pravilo iz različitih smjerova: prvi provjerava jesu li svi uvjeti zadovoljeni, a drugi postoji li barem jedan razlog za dodatnu provjeru.

Za provjeru raspona možemo napisati `bodovi >= 0 and bodovi <= 100` ili kraće `0 <= bodovi <= 100`. Drugi oblik je ulančana usporedba: provjerava je li vrijednost istodobno između donje i gornje granice. Granice su uključene zbog operatora `<=`.

Raspon uvijek treba čitati zajedno s operatorima. Izraz `0 < bodovi < 100` isključuje nulu i sto, dok ih `0 <= bodovi <= 100` uključuje. Python ulančanu usporedbu tumači kao povezane usporedbe iste vrijednosti, a ne kao postupni izračun s lijeva na desno.

Pogledajte sljedeći program. Student ima `45` bodova. Je li uvjet pravilno definiran, ako nije što treba promijeniti i zašto?

```
bodovi = 45

if bodovi == 50 or 60:
    print("Bodovi iznose 50 ili 60.")
```

▼ Prikaži provjeru i objašnjenje

Program ispisuje poruku iako vrijednost varijable `bodovi` iznosi `45`:

Bodovi iznose 50 ili 60.

⚠ Svaku mogućnost zapišite kao potpunu provjeru

Zapis `bodovi == 50 or 60` ne provjerava iznose li bodovi `50` ili `60`. Broj `60` u tom izrazu nije usporedba, nego samostalna vrijednost koja se tumači kao istinita. Zato cijeli uvjet daje istinit rezultat neovisno o vrijednosti varijable `bodovi`.

Svaku mogućnost treba zapisati kao potpunu provjeru:

```
bodovi = 45

if bodovi == 50 or bodovi == 60:
    print("Bodovi iznose 50 ili 60.")
```

U ispravljenom programu obje strane operatora `or` predstavljaju usporedbe. Za `bodovi = 45` obje daju `False`, pa se poruka ne ispisuje.

3.2 Provjera unutar druge provjere

Prije tumačenja bodova korisno je utvrditi pripadaju li uopće dopuštenom rasponu. Rezultat `145` ne želimo proglasiti izvrsnim ako je najveći mogući broj bodova `100`. Najprije provjeravamo valjanost, a tek zatim obrađujemo prihvatljivu vrijednost.

Jedan složeni uvjet može odgovoriti samo na zajedničko pitanje:

```
bodovi = 145

if 0 <= bodovi <= 100 and bodovi >= 60:
    print("Prag je zadovoljen.")
else:
    print("Prag nije zadovoljen.")
```

Za vrijednost `145` program tvrdi da prag nije zadovoljen, iako je stvarni problem nevaljan podatak. Iz završne poruke ne možemo razlikovati rezultat ispod praga od vrijednosti izvan mogućeg raspona. Zato odluke trebamo izvesti redom: prvo utvrditi smijemo li podatak obrađivati, a zatim provjeriti prag.

💡 Ugniježđeno grananje

Ugniježđeno grananje je grananje smješteno unutar bloka drugog grananja. Unutarnji uvjet provjerava se samo ako izvršavanje dođe do njega, odnosno ako je odabrana odgovarajuća vanjska grana.

```
bodovi = 74

if 0 <= bodovi <= 100:
    if bodovi >= 60:
        print("Prag je zadovoljen.")
    else:
        print("Prag nije zadovoljen.")
else:
    print("Bodovi moraju biti između 0 i 100.")
```

▼ Prikaži ispis i objašnjenje

Prag je zadovoljen.

Vanjski uvjet potvrđuje da je 74 prihvatljiva vrijednost. Tek tada provjeravamo prag. Za 145 preskočilo bi se cijelo unutarnje grananje i ispisala poruka o rasponu.

Unutarnji `if` i njegov `else` imaju četiri razmaka uvlačenja, njihove naredbe osam. Vanjski `else` poravnat je s vanjskim `if`. Tako se vidi kojem grananju pripada svaka alternativa.

Program ima tri moguća puta:

Vrijednost	Vanjska provjera raspona	Unutarnja provjera praga	Ispis
145	False	Ne provjerava se	Bodovi nisu u dopuštenom rasponu
45	True	False	Prag nije zadovoljen
74	True	True	Prag je zadovoljen

Unutarnje grananje postoji samo na putu na kojem je podatak valjan. To je glavna razlika u odnosu na jedan veliki izraz: struktura ne daje samo završni odgovor nego prikazuje redoslijed odluka.

3.3 Odabir između složenog uvjeta i ugniježđenog grananja

Ako trebamo samo jednu odluku koja ovisi o dvama uvjetima, `and` je često pregledniji. Ako nakon prve provjere trebamo različite poruke ili dodatnu obradu, ugniježđeno grananje može jasnije prikazati korake.

U primjeru s bodovima vrijednosti 45 i 145 ne smiju se obrađivati jednako. Broj 45 valjan je rezultat koji ne doseže prag, dok je 145 nevaljan jer je izvan dopuštenog raspona. Složeni uvjet `0 <= bodovi <= 100 and bodovi >= 60` za obje vrijednosti daje `False`. Kada bismo uz njega imali samo jednu granu `else`, oba bi

slučaja završila u istoj grani i dobila istu poruku. Ugniježđeno grananje omogućuje da prvo odvojimo nevaljane podatke, a zatim među valjanima provjerimo je li prag zadovoljen.

Zadatak

Za rezervaciju opreme potrebno je imati aktivan korisnički račun i barem jedan slobodan uređaj. Zadane su vrijednosti `racun_aktivan = True` i `slobodni_uredaji = 0`. Program treba razlikovati tri slučaja: račun nije aktivan, račun je aktivan ali opreme nema te rezervacija je moguća. Kako biste dobili odgovarajuću poruku za svaki slučaj?

▼ Prikaži moguće rješenje i ispis

```
racun_aktivan = True
slobodni_uredaji = 0

if racun_aktivan:
    if slobodni_uredaji > 0:
        print("Rezervacija je moguća.")
    else:
        print("Trenutačno nema slobodne opreme.")
else:
    print("Korisnički račun nije aktivan.")
```

Trenutačno nema slobodne opreme.

Provjerite i neaktivan račun te aktivan račun s dva slobodna uređaja. Moguće je napisati i povezano grananje koje najprije provjerava neaktivan račun, zatim nedostatak opreme, a završnim `else` potvrđuje rezervaciju.

4 for petlja

4.1 Zašto nam treba petlja

Pretpostavimo da želimo ispisati svaki znak oznake u zasebnom retku. Za oznaku od tri znaka možemo zapisati tri ispisa:

```
oznaka = "A17"
print(oznaka[0])
print(oznaka[1])
print(oznaka[2])
```

Što biste trebali promijeniti kada oznaka ima deset znakova? Što ako je svaki put unosi korisnik pa joj unaprijed ne znamo duljinu? Postupak nad svakim znakom ostaje isti, mijenja se samo znak koji obrađujemo.

Petlja i iteracija

Petlja je upravljačka struktura koja omogućuje ponavljanje bloka naredbi.

Iteracija je jedno izvršavanje tijela petlje.

Petlja `for` redom preuzima vrijednosti iz objekta kroz koji možemo prolaziti, primjerice znakove teksta, i za svaku izvršava svoje tijelo.

Osnovni zapis petlje `for` izgleda ovako:

```
for vrijednost in izvor:
    naredbe_koje_se_ponavljaju

naredbe_nakon_petlje
```

Dio zapisa	Značenje
<code>for</code>	Označava početak petlje
<code>vrijednost</code>	Varijabla kojoj Python u svakoj iteraciji dodjeljuje sljedeći element
<code>in</code>	Povezuje varijablu petlje s izvorom vrijednosti
<code>izvor</code>	Objekt kroz koji prolazimo, u prvom primjeru to je tekst
<code>:</code> i uvučeni redci	Označavaju tijelo koje se ponavlja

Python ne izvršava petlju „odjednom”. Uzima prvi element, dodjeljuje ga varijabli petlje, izvršava cijelo tijelo, zatim uzima sljedeći element i ponavlja postupak. Tek nakon što potroši sve elemente prelazi na naredbu izvan petlje.

```
oznaka = "A17"

for znak in oznaka:
    print(znak)

print("Obrada oznake je završena.")
```

▼ Prikaži ispis i tijek izvršavanja

```
A
1
7
Obrada oznake je završena.
```

Iteracija	Vrijednost varijable <code>znak</code>	Izvršena naredba
Prva	"A"	Ispisuje <code>A</code>
Druga	"1"	Ispisuje <code>1</code>
Treća	"7"	Ispisuje <code>7</code>

Kada više nema znakova, petlja završava. Posljednja poruka nalazi se izvan petlje i ispisuje se jednom.

4.2 Varijabla petlje i tijelo petlje

U zapisu `for znak in oznaka:` ime `znak` predstavlja varijablu petlje. Python joj prije svake iteracije dodjeljuje sljedeću vrijednost. Ne moramo je unaprijed definirati. Ime biramo sami, kao i bilo kojoj drugoj varijabli, `znak` je čitljivije od proizvoljne kratice kada obrađujemo znakove.

Za tekst `"A17"` izvor ima tri elementa jer se sastoji od tri znaka. Varijabla `znak` najprije sadrži `"A"`, zatim `"1"`, a na kraju `"7"`. Svaka je od tih vrijednosti tekst duljine jedan. Python ne pokušava znakove `"1"` i `"7"` automatski pretvoriti u brojeve.

U petlji `for znak in oznaka` riječ `in` označava iz kojeg izvora Python redom uzima vrijednosti: u ovom slučaju uzima znakove iz varijable `oznaka`. Riječ `in` možemo koristiti i izvan petlje. Dakle izraz `"A" in oznaka` postavlja pitanje sadrži li tekst spremljen u varijabli `oznaka` znak `"A"` te vraća `True` ili `False`.

Tijelo petlje može imati više naredbi. Sve se izvršavaju za trenutnu vrijednost prije prelaska na sljedeću. Python ne izvršava prvo prvi redak za sve znakove, a zatim drugi redak za sve znakove.

Uvlačenje ima isto značenje kao kod grananja: pokazuje koje naredbe pripadaju petlji. Naredba vraćena na početnu razinu više se ne ponavlja. Ako završni izvještaj slučajno ostavimo uvučen, dobit ćemo ga nakon svake iteracije umjesto jednom nakon cijele obrade.

```
tekst = "Ab"

for znak in tekst:
    print("Izvorni znak:", znak)
    print("Veliko slovo:", znak.upper())

print("Izvorni tekst:", tekst)
```

▼ Prikaži očekivani ispis

```
Izvorni znak: A
Veliko slovo: A
Izvorni znak: b
Veliko slovo: B
Izvorni tekst: Ab
```

Metoda `upper()` vraća novi tekst. Njezino korištenje pri ispisu ne mijenja izvorni tekst ni varijablu `znak`.

4.3 Petlja bez iteracija

Ako je izvorni tekst prazan, nema znakova koje bi petlja preuzela. Tijelo se tada ne izvršava nijednom. Program svejedno nastavlja nakon petlje. To je očekivano ponašanje, a ne pogreška.

Ovaj slučaj treba uzeti u obzir kada se rezultat stvara samo unutar petlje. Ako se varijabla prvi put definira u tijelu, kod praznog izvora ta se dodjela nikada neće izvršiti. Zato brojače, zbrojeve i druge početne vrijednosti najčešće postavljamo prije petlje.

```
tekst = ""

for znak in tekst:
    print("Znak:", znak)

print("Gotovo.")
```

▼ Prikaži očekivani ispis

```
Gotovo.
```

⚠ Varijabla petlje nije automatski redni broj

Pri prolasku kroz tekst varijabla petlje sadrži znak, a ne njegov indeks. U primjeru `for znak in "A17":` vrijednosti su `"A"`, `"1"` i `"7"`, sve tipa `str`. Vrijednosti ovise o objektu kroz koji prolazimo.

🚧 Pregled novih naredbi

Naredba

Namjena

```
for vrijednost in izvor:
```

Izvršava tijelo petlje za svaku vrijednost preuzetu iz izvora

Zadatak

Oznaka uređaja zapisana je kao " b7k ". Pripremite oznaku bez rubnih razmaka i s velikim slovima, a zatim svaki njezin znak ispišite u zasebnom retku unutar uglatih zagrada. Na kraju jednom ispišite cijelu uređenu oznaku. Kako bi se program ponašao za oznaku koja sadrži samo razmake?

▼ Prikaži moguće rješenje i ispis

```
izvorna_oznaka = " b7k "  
oznaka = izvorna_oznaka.strip().upper()  
  
for znak in oznaka:  
    print "[" + znak + "]"  
  
print("Uređena oznaka:", oznaka)
```

[B]

[7]

[K]

Uređena oznaka: B7K

Ako su u izvornom tekstu samo razmaci, nakon `strip()` ostaje prazan tekst. Petlja nema iteracija, a završni ispis svejedno se izvršava.

5 Određivanje raspona funkcijom range

5.1 Ponavljanje zadani broj puta

Petlja `for` može obrađivati znakove teksta, ali često trebamo ponoviti postupak određeni broj puta ili proći kroz niz cijelih brojeva. Primjerice, želimo ispisati oznake pet mjerenja ili izračunati nešto za svaki redni broj.

Petlju već možemo napisati ako sami navedemo tekst koji sadrži potrebne vrijednosti:

```
for redni_broj in "12345":
    print("Mjerenje", redni_broj)
```

Ovdje su redni brojevi znakovi tipa `str`, a ne brojevi s kojima možemo računati. Za pedeset mjerenja morali bismo ručno sastaviti mnogo dulji zapis, pri čemu višeznamenasti brojevi više ne bi bili jedan element. Potreban nam je način da Python prema zadanim granicama pripremi cijele brojeve kroz koje će petlja prolaziti. Za to koristimo funkciju `range()`.

Raspon cijelih brojeva

`range()` stvara objekt koji predstavlja niz cijelih brojeva određen početkom, završnom granicom i korakom. Početna vrijednost uključena je u raspon, a završna granica nije. `range()` nije lista i ne stvara unaprijed zasebnu listu svih svojih brojeva.

Funkcija može primiti jedan, dva ili tri argumenta. Broj argumenata određuje koje dijelove raspona navodimo, a koje Python postavlja prema zadanim pravilima:

Zapis	Početak	Završna granica	Korak
<code>range(kraj)</code>	<code>0</code>	<code>kraj</code>	<code>1</code>
<code>range(pocetak, kraj)</code>	<code>pocetak</code>	<code>kraj</code>	<code>1</code>
<code>range(pocetak, kraj, korak)</code>	<code>pocetak</code>	<code>kraj</code>	<code>korak</code>

Raspon možemo zamisliti kao pravilo za dobivanje sljedeće vrijednosti. Python počinje od uključene početne vrijednosti, nakon svake iteracije dodaje korak i zaustavlja se prije nego što bi dosegnuo ili prešao isključenu završnu granicu u smjeru kretanja.

Kada navedemo samo jedan argument, on označava završnu granicu. Početak je `0`, a korak `1`.

```
for broj in range(4):
    print(broj)
```

▼ Prikaži ispis i objašnjenje

```
0
1
2
3
```

Petlja ima četiri iteracije, iako je posljednja vrijednost 3. Broj iteracija i vrijednost varijable u posljednjoj iteraciji nisu isto. Vrijednost 4 završna je granica i zato se ne preuzima.

To možemo pratiti kao niz odluka: početna vrijednost je 0 i nalazi se prije granice 4, pa ulazi u petlju. Nakon nje slijede 1, 2 i 3. Sljedeća bi vrijednost bila 4, ali ona je jednaka završnoj granici i tu raspon završava.

5.2 Početak i završna granica

Kada želimo početi od nekog drugog broja, navodimo dva argumenta: `range(pocetak, kraj)`. To je korisno za redne brojeve koji obično počinju od 1.

U zapisu `range(1, 5)` prvi argument govori odakle počinjemo, a drugi gdje se zaustavljamo. Dobivamo 1, 2, 3 i 4. Početak se pojavljuje u rasponu, završna vrijednost 5 ne pojavljuje se.

```
for redni_broj in range(1, 5):  
    print(f"Mjerenje {redni_broj}")
```

▼ Prikaži očekivani ispis

```
Mjerenje 1  
Mjerenje 2  
Mjerenje 3  
Mjerenje 4
```

Za brojeve od 1 do uključivo n koristimo `range(1, n + 1)`. Dodatak 1 ne proširuje tijelo petlje niti mijenja vrijednost n, njime postavljamo prvu isključenu vrijednost neposredno nakon posljednje koju želimo. Ne treba ga dodavati automatski u svaki `range()`: prvo treba zapisati željene vrijednosti i tek zatim iz njih odrediti početak, isključenu granicu i korak.

5.3 Korak i odbrojanje

Treći argument određuje razliku između uzastopnih vrijednosti. U `range(2, 9, 2)` krećemo od 2, svaki put dodajemo 2 i stajemo prije 9. Negativan korak koristimo kada vrijednosti trebaju opadati.

Korak ne znači „uzmi svaki drugi element neke postojeće liste”, nego određuje kako se iz trenutnog broja dobiva sljedeći. Pozitivan korak povećava vrijednost, a negativan je smanjuje. Završna granica i dalje se ne uključuje.

```
print("Parni redni brojevi:")  
for broj in range(2, 9, 2):  
    print(broj)  
  
print("Odbrojanje:")  
for broj in range(3, 0, -1):  
    print(broj)
```

▼ Prikaži očekivani ispis

Parni redni brojevi:

2

4

6

8

Odbrojavanje:

3

2

1

Kod odbrojavanja `0` je isključena završna granica. Ako želimo ispisati i nulu, granicu treba pomaknuti na `-1`.

5.4 Prazan raspon i ograničenja

Smjer koraka mora odgovarati željenom kretanju. U `range(5, 2)` zadani korak je pozitivan. Od početka `5` ne možemo rastom doći do vrijednosti manje od završne granice `2`, pa je raspon prazan. Python tada ne javlja pogrešku: petlja jednostavno nema iteracija.

Prazan raspon može biti namjeran, ali često otkriva pogrešno postavljene granice ili korak. Ako očekujete ispis, a petlja ne napravi ništa, provjerite početak, kraj i smjer koraka. Za rastući raspon početak je obično manji od kraja, a za padajući raspon veći.

Zapis	Vrijednosti koje petlja preuzima
<code>range(3)</code>	<code>0, 1, 2</code>
<code>range(3, 6)</code>	<code>3, 4, 5</code>
<code>range(1, 8, 3)</code>	<code>1, 4, 7</code>
<code>range(4, -1, -2)</code>	<code>4, 2, 0</code>
<code>range(5, 2)</code>	Nema vrijednosti
<code>range(2, 2)</code>	Nema vrijednosti

⚠ Argumenti funkcije range

U ovoj vježbi argumenti funkcije `range()` moraju biti cijeli brojevi. Decimalni korak poput `0.5` nije podržan, a korak `0` nije dopušten. Primjerice, `range(0, 5, 0)` izaziva `ValueError`. Negativan korak ne odabire se automatski kada je početak veći od kraja: morate ga navesti.

🚧 Pregled novih funkcija

Funkcija

Namjena

`range(kraj)`

Predstavlja cijele brojeve od nule do isključene završne granice

`range(pocetak, kraj)`

Omogućuje zadavanje početne vrijednosti

`range(pocetak, kraj, korak)`

Omogućuje zadavanje smjera i veličine koraka

Zadatak

Mjerenja se izvode u minutama 0, 5, 10, 15 i 20. Za svaki termin ispišite poruku poput `Mjerenje u 5. minuti`. Nakon rasporeda ispišite odbrojavanje od 4 do 1 i zatim jednom poruku `Početak mjerenja`. Kako biste odredili početak, kraj i korak za oba dijela?

▼ Prikaži moguće rješenje i ispis

```
for minuta in range(0, 21, 5):  
    print(f"Mjerenje u {minuta}. minuti")  
  
for broj in range(4, 0, -1):  
    print(broj)  
  
print("Početak mjerenja")
```

```
Mjerenje u 0. minuti  
Mjerenje u 5. minuti  
Mjerenje u 10. minuti  
Mjerenje u 15. minuti  
Mjerenje u 20. minuti  
4  
3  
2  
1  
Početak mjerenja
```

6 Izračuni i provjere unutar petlje

6.1 Zbrajanje vrijednosti kroz više iteracija

Petlja nije korisna samo za ispis. Može postupno graditi rezultat: ukupan trošak, zbroj mjerenja ili broj prihvaćenih odgovora. Da bismo sačuvali rezultat prethodnih iteracija, koristimo varijablu definiranu prije petlje i mijenjamo je tijekom ponavljanja.

Za četiri unaprijed poznata broja zbroj možemo napisati izravno:

```
print(1 + 2 + 3 + 4)
```

Takav zapis ne koristi petlju i morao bi se ručno proširivati svaki put kada se promijeni gornja granica. Ako petlja redom daje vrijednosti od `1` do `100`, u svakoj iteraciji vidimo samo trenutni broj. Potrebna nam je varijabla koja će pamtit rezultati svih prethodnih iteracija.

Akumulator i brojač

Akumulator je varijabla u kojoj postupno prikupljamo rezultat, primjerice zbroj obrađenih vrijednosti.

Brojač je varijabla kojom pratimo broj pojavljivanja ili događaja, najčešće povećanjem za `1`. To su uloge običnih varijabli, a ne posebni Pythonovi tipovi podataka.

Kod takvog izračuna razlikujemo tri mjesta u programu:

Mjesto	Svrha	Primjer za zbroj
Prije petlje	Postavlja početnu vrijednost prije obrade prvog podatka	<code>zbroj = 0</code>
U tijelu petlje	Uključuje trenutnu vrijednost u dosadašnji rezultat	<code>zbroj += broj</code>
Nakon petlje	Koristi konačan rezultat kada su sve vrijednosti obrađene	<code>print(zbroj)</code>

Akumulator mora postojati prije prve iteracije jer se pri ažuriranju koristi njegova prethodna vrijednost. Nakon svake iteracije ista varijabla sadrži novi međurezultat, koji postaje početna točka sljedeće iteracije.

Zadatak: Izračunajte zbroj cijelih brojeva od `1` do `4`.

```
zbroj = 0

for broj in range(1, 5):
    zbroj += broj
    print("Dodano:", broj, "Trenutačni zbroj:", zbroj)

print("Konačni zbroj:", zbroj)
```

▼ Prikaži ispis i promjene vrijednosti

Dodano: 1 Trenutačni zbroj: 1
Dodano: 2 Trenutačni zbroj: 3
Dodano: 3 Trenutačni zbroj: 6
Dodano: 4 Trenutačni zbroj: 10
Konačni zbroj: 10

Vrijednost broj	zbroj prije dodavanja	zbroj nakon dodavanja
1	0	1
2	1	3
3	3	6
4	6	10

Zapis `zbroj += broj` znači `zbroj = zbroj + broj`. Desna strana koristi dotadašnji zbroj, a rezultat ga zamjenjuje. Početna nula prikladna je za zbrajanje jer ne mijenja prvi dodani broj.

Važno je razlikovati varijable `broj` i `zbroj`. Varijabla `broj` u svakoj iteraciji dobiva novu vrijednost iz raspona. Varijabla `zbroj` ne dolazi iz raspona, ona čuva sve što je prikupljeno do tog trenutka. Nakon završetka petlje posljednja vrijednost varijable `broj` bila je `4`, ali konačni `zbroj` iznosi `10`.

⚠ Početnu vrijednost postavite prije petlje

Ako `zbroj = 0` stavite unutar tijela petlje, svaka će iteracija obrisati dotadašnji zbroj. Program se može izvršiti bez pogreške, ali više neće zbrajati sve vrijednosti. Isto vrijedi za brojač koji se treba povećavati kroz više iteracija.

6.2 Brojanje vrijednosti koje zadovoljavaju uvjet

Za brojanje događaja povećavamo brojač samo kada se događaj dogodi. Primjerice, u tekstu možemo brojiti pojavljivanja znaka `?`. Petlja omogućuje pregled svakog znaka, a grananje odlučuje hoće li se taj znak uračunati.

Brojač počinje od nule jer prije obrade još nismo pronašli nijedno pojavljivanje. U svakoj iteraciji postavljamo pitanje odnosi li se trenutna vrijednost na događaj koji brojimo. Ako je odgovor potvrđan, povećavamo brojač za jedan. Ako nije, brojač ostaje nepromijenjen.

```
tekst = "Da? Ne? Možda."  
broj_upitnika = 0  
  
for znak in tekst:  
    if znak == "?":  
        broj_upitnika += 1  
  
print("Broj upitnika:", broj_upitnika)
```

▼ Prikaži ispis i objašnjenje

Broj upitnika: 2

Petlja prolazi kroz sve znakove, ali brojač se povećava samo dvaput. Naredba povećavanja pripada unutarnjem grananju pa ima dvije razine uvlačenja. Ovaj rezultat mogli bismo dobiti i poznatom metodom `count()`, ovdje razrađujemo postupak brojanja koji poslije možemo primijeniti na složenije uvjete.

Za dio teksta `"Da?"` promjene izgledaju ovako:

Znak	Je li <code>znak == "?"</code> ?	Brojač nakon iteracije
<code>"D"</code>	Ne	0
<code>"a"</code>	Ne	0
<code>"?"</code>	Da	1

Brojač se ne povećava u svakoj iteraciji. Njegovo ažuriranje nalazi se unutar `if` upravo zato što želimo brojiti samo odabrane vrijednosti.

6.3 Unos više vrijednosti i računanje prosjeka

Ako korisnik treba unijeti tri mjerenja, ne moramo pisati tri odvojena poziva `input()`. Poziv možemo smjestiti u petlju. Svaka iteracija tada traži novi podatak, pretvara ga u broj i dodaje u zbroj.

Prosjeak zahtijeva dvije informacije: zbroj vrijednosti i njihov broj. U ovom primjeru broj je unaprijed poznat i spremljen u `broj_mjerenja`. Zbroj nastaje postupno u petlji. Dijeljenje nema smisla prije završetka unosa jer tada još nemamo sve vrijednosti koje trebaju sudjelovati u prosjeku.

```
broj_mjerenja = 3
zbroj = 0.0

for redni_broj in range(1, broj_mjerenja + 1):
    mjerenje = float(input(f"Mjerenje {redni_broj}: "))
    zbroj += mjerenje

prosjeak = zbroj / broj_mjerenja
print(f"Prosjeak: {prosjeak:.2f}")
```

▼ Prikaži primjer unosa i rezultat

Mjerenje 1: 18
Mjerenje 2: 21
Mjerenje 3: 24
Prosjeak: 21.00

Varijabla `mjerenje` svaki put dobiva novu vrijednost. Ne čuvamo sva pojedinačna mjerenja, nego njihov zbroj. To je dovoljno za ovaj izračun. Prosjek računamo nakon petlje, kada su unesena sva tri podatka.

Za prikazane unose zbroj se mijenja iz `0.0` u `18.0`, zatim `39.0` i naposljetku `63.0`. Nakon treće iteracije petlja završava, a izraz `63.0 / 3` daje prosjek `21.0`. Oblikovanje `:.2f` mijenja način ispisa u `21.00`, ali sama varijabla `prosjek` ostaje broj.

U ovom primjeru pretpostavljamo ispravan brojčani unos s decimalnom točkom i pozitivan broj mjerenja. Ako bismo broj mjerenja dopustili postaviti na `0`, završno dijeljenje izazvalo bi pogrešku.

6.4 Početna vrijednost ovisi o izračunu

Nula je prikladna za zbroj, ali nije prikladna za svaki izračun. Ako množimo vrijednosti, početak od nule zadržao bi umnožak na nuli bez obzira na ostale brojeve. Zato umnožak započinjemo jedinicom.

Početna vrijednost treba biti neutralna za operaciju koju ponavljamo. Dodavanje nule ne mijenja broj, pa zbroj počinje nulom. Množenje jedinicom ne mijenja broj, pa umnožak počinje jedinicom. Početnu vrijednost ne biramo zato što je „prva”, nego zato što omogućuje da prvi obrađeni podatak ispravno uđe u rezultat.

```
umnozak = 1

for broj in range(1, 5):
    umnozak *= broj

print("Umnožak:", umnozak)
```

▼ Prikaži ispis i objašnjenje

Umnožak: 24

Vrijednost se mijenja redom u `1`, `2`, `6` i `24`. Izračun `1 * 2 * 3 * 4` ujedno je faktoriyel broja `4`, koji se matematički označava s `4!`. Za razumijevanje petlje važno je pratiti uzastopno množenje, a ne pamtiti naziv tog izračuna.

Zadatak

Za svaki cijeli broj od `1` do `15` provjerite je li djeljiv s `3`. Program treba ispisati takve brojeve, izračunati koliko ih ima te koliki je njihov zbroj. Završni broj pronađenih vrijednosti i zbroj ispišite jednom, nakon obrade svih brojeva.

▼ Prikaži moguće rješenje i ispis

```
brojac = 0
zbroj = 0

for broj in range(1, 16):
    if broj % 3 == 0:
        print(broj)
        brojac += 1
        zbroj += broj

print("Broj vrijednosti:", brojac)
print("Zbroj:", zbroj)
```

```
3
6
9
12
15
Broj vrijednosti: 5
Zbroj: 45
```

Provjera ostatka pri dijeljenju određuje pripada li broj traženoj skupini. Brojač povećavamo za jedan, a zbroj za samu vrijednost broja. Te dvije promjene imaju različitu svrhu.

7 while petlja

7.1 Ponavljanje dok je uvjet zadovoljen

Kod unosa triju mjerenja unaprijed smo znali koliko podataka treba obraditi. No koliko će puta korisnik pogrešno upisati dogovorenu oznaku? To ne možemo znati unaprijed. Možemo odrediti uvjet pod kojim treba nastaviti tražiti novi unos.

Možemo unaprijed napisati nekoliko pokušaja:

```
grupa = input("Unesite grupu A ili B: ").strip().upper()

if grupa != "A" and grupa != "B":
    grupa = input("Pokušajte ponovno: ").strip().upper()

if grupa != "A" and grupa != "B":
    grupa = input("Pokušajte ponovno: ").strip().upper()
```

Ovaj program ponavlja isti kod i nakon trećeg pogrešnog unosa ipak nastavlja s neprihvatljivom vrijednošću. Ne znamo koliko će pokušaja trebati, ali znamo da novi unos treba tražiti sve dok oznaka nije valjana. Takav uvjet može upravljati ponavljanjem.

Petlja while

Petlja `while` ponavlja tijelo dok se njezin uvjet tumači kao istinit. Uvjet se provjerava prije svakog izvršavanja tijela, uključujući prvo. Kada postane lažan, izvršavanje se nastavlja nakon petlje.

Osnovni zapis izgleda ovako:

```
pocetno_stanje

while uvjet:
    naredbe_koje_se_ponavljaju
    promjena_stanja

naredbe_nakon_petlje
```

Izrazi `pocetno_stanje`, `uvjet` i `promjena_stanja` predstavljaju uloge u postupku, a ne posebne Pythonove naredbe. Početno stanje mora postojati prije prve provjere. Uvjet određuje treba li napraviti novu iteraciju. Tijelo obavlja posao, a promjena stanja približava program završetku.

Petlja `while` radi po obrascu „provjeri pa izvrši”. Zbog toga se tijelo može izvršiti mnogo puta, jednom ili nijednom. Nakon svake iteracije Python se vraća na zaglavlje i ponovno izračunava uvjet s trenutačnim vrijednostima varijabli.

Prvo ćemo tijek promatrati na kratkom primjeru s brojačem:

```

preostalo = 3

while preostalo > 0:
    print("Preostalo:", preostalo)
    preostalo -= 1

print("Gotovo.")

```

▼ Prikaži ispis i tijek provjera

Preostalo: 3
 Preostalo: 2
 Preostalo: 1
 Gotovo.

Vrijednost pri provjeri	Rezultat <code>preostalo > 0</code>	Što slijedi?
3	True	Ispisuje 3, zatim smanjuje na 2
2	True	Ispisuje 2, zatim smanjuje na 1
1	True	Ispisuje 1, zatim smanjuje na 0
0	False	Preskače tijelo i izlazi iz petlje

Uvjet se provjerio četiri puta, a tijelo se izvršilo triput. Posljednja provjera služi utvrđivanju da više ne treba ponavljati.

7.2 Početna vrijednost, promjena i završetak

Kod petlje `while` moramo jasno odrediti početno stanje, uvjet ponavljanja i promjenu koja omogućuje završetak. Za razliku od petlje `for`, `while` ne preuzima automatski sljedeću vrijednost iz izvora.

U primjeru s odbrojavanjem ta su tri dijela lako vidljiva:

Dio postupka	Kod	Uloga
Početno stanje	<code>preostalo = 3</code>	Određuje vrijednost prije prve provjere
Uvjet	<code>preostalo > 0</code>	Dopušta novu iteraciju dok postoji nešto za odbrojiti
Promjena	<code>preostalo -= 1</code>	Smanjuje vrijednost kako bi uvjet naposljetku postao lažan

Promjena ne mora uvijek biti povećavanje ili smanjivanje broja. Kod korisničkog unosa stanje se mijenja novim tekstom, a kod obrade zalihe oduzimanjem potrošene količine. Bitno je da možemo objasniti kako program može dosegnuti završetak.

Ako u prethodnom primjeru početnu vrijednost postavimo na `0`, uvjet je odmah lažan. Program tada ispisuje samo završnu poruku. Ako uklonimo `preostalo -= 1`, vrijednost zauvijek ostaje `3`, pa uvjet ne prestaje vrijediti.

I promjena u pogrešnom smjeru može stvoriti beskonačnu petlju. Ako bismo uz uvjet `preostalo > 0` napisali `preostalo += 1`, vrijednost bi rasla i ostajala pozitivna. Sintaksa bi bila ispravna, ali program se ne bi približavao završetku.

Beskonačna petlja

Beskonačna petlja nema dostižan završetak tijekom uobičajenog izvršavanja. Čest uzrok je izostavljena promjena varijable u uvjetu ili promjena u pogrešnom smjeru. Ako se program ne zaustavlja, kliknite u terminal i pritisnite `Ctrl+C` za prekid programa.

7.3 Ponavljanje korisničkog unosa

Sljedeći program traži oznaku grupe sve dok korisnik ne upiše `A` ili `B`. Unos prvo ujednačava uklanjanjem rubnih razmaka i pretvaranjem slova u velika. Tako `" a "` i `"A"` predstavljaju isti prihvatljiv unos.

Prvi unos nalazi se prije petlje jer uvjet ne možemo provjeriti dok varijabla `grupa` nema vrijednost. Ako je prvi unos odmah valjan, tijelo se neće izvršiti nijednom. Ako nije, tijelo ispisuje objašnjenje i traži novu vrijednost. Taj novi unos važan je jer mijenja stanje koje će se provjeriti na početku sljedeće iteracije.

```
grupa = input("Unesite grupu A ili B: ").strip().upper()

while grupa != "A" and grupa != "B":
    print("Oznaka grupe nije prihvatljiva.")
    grupa = input("Unesite grupu A ili B: ").strip().upper()

print("Odabrana grupa:", grupa)
```

▼ **Prikaži primjer unosa i ispisa**

```
Unesite grupu A ili B: C
Oznaka grupe nije prihvatljiva.
Unesite grupu A ili B: b
Odabrana grupa: B
```

Prvi unos osigurava vrijednost za prvu provjeru. Novi unos unutar petlje može promijeniti rezultat sljedeće provjere. Ponavljamo dok oznaka nije ni `A` ni `B`, zato koristimo `and`. Uvjet s `or` bio bi uvijek istinit: čak i `A` nije jednako `B`.

Provjerimo dopuštene vrijednosti pojedinačno. Za grupu `"A"` izraz `grupa != "A"` je `False`, a `grupa != "B"` je `True`. Njihovo povezivanje operatorom `and` daje `False`, pa petlja završava. Kod operatora `or` rezultat bi bio `True` zbog druge usporedbe i program bi pogrešno odbio valjanu oznaku `"A"`. Isto se

simetrično događa za "B".

Ovdje provjeravamo tekstualnu oznaku, pa nema pretvorbe koja bi mogla propasti zbog nebrojčanog unosa. Kod brojčanih primjera `int()` ili `float()` još mogu javiti pogrešku ako korisnik upiše neprikladan tekst. Provjera brojčanog raspona i provjera može li se tekst pretvoriti u broj dva su odvojena koraka.

7.4 Odabir između `for` i `while`

Isti se problem ponekad može riješiti s obje petlje. Važno je odabrati zapis iz kojeg se jasno vidi zašto ponavljamo i kada završavamo.

Kod petlje `for` izvor vrijednosti određuje prirodan slijed iteracija. Python preuzima sljedeći znak ili sljedeći broj iz raspona. Kod petlje `while` nema takvog automatskog slijeda: nova iteracija postoji zato što je uvjet još uvijek istinit. Zato pri čitanju `while` petlje uvijek trebamo pronaći varijable iz uvjeta i mjesta na kojima se one mijenjaju.

Situacija	Uobičajen odabir i razlog
Obrada svakog znaka teksta	<code>for</code> : svaki znak preuzimamo redom
Točno pet korisničkih unosa	<code>for</code> s <code>range()</code> : broj ponavljanja je zadan
Unos dok korisnik ne odabere dopuštenu oznaku	<code>while</code> : završetak ovisi o uvjetu
Ponavljanje dok se ne potroši zaliha	<code>while</code> : pratimo promjenjivo stanje

Petlja `for` ne zahtijeva da programer uvijek unaprijed zna broj elemenata, bitno je da postoji izvor kroz koji prolazi. Petlja `while` može se koristiti i za poznat broj ponavljanja, ali tada sami održavamo brojač.

Odabir petlje ne mijenja problem koji rješavamo, ali utječe na čitljivost i mogućnost pogreške. Za četiri ponavljanja `for redni_broj in range(1, 5):` izravno prikazuje željene vrijednosti. Isti postupak s `while` zahtijeva početnu vrijednost, uvjet i ručno povećavanje brojača, pa postoji više mjesta na kojima možemo pogriješiti.

Pregled novih naredbi

Naredba	Namjena
<code>while uvjet:</code>	Ponavlja tijelo dok je uvjet istinit

Zadatak

Za slanje jedne pošiljke potrebno je 6 naljepnica, a na raspolaganju ih je 23. Dok imate dovoljno naljepnica, pripremite novu pošiljku i smanjite zalihu. Nakon svake pripreme ispišite redni broj pošiljke i preostalu zalihu. Na kraju ispišite koliko je pošiljaka pripremljeno i koliko je naljepnica ostalo.

▼ Prikaži moguće rješenje i ispis

```
naljepnice = 23
po_posiljci = 6
broj_posiljaka = 0

while naljepnice >= po_posiljci:
    naljepnice -= po_posiljci
    broj_posiljaka += 1
    print(f"Pošiljka {broj_posiljaka}, preostalo naljepnica: {naljepnice}")

print("Pripremljeno pošiljaka:", broj_posiljaka)
print("Preostalo naljepnica:", naljepnice)
```

```
Pošiljka 1, preostalo naljepnica: 17
Pošiljka 2, preostalo naljepnica: 11
Pošiljka 3, preostalo naljepnica: 5
Pripremljeno pošiljaka: 3
Preostalo naljepnica: 5
```

Uvjet `>=` dopušta pripremu i kada zaliha točno odgovara jednoj pošiljci. Da smo provjeravali samo `naljepnice > 0`, mogli bismo pripremiti pošiljku za koju nema dovoljno naljepnica i dobiti negativnu zalihu.

8 Prekidanje petlje i preskakanje iteracije

Dosadašnje petlje obrađivale su sve predviđene vrijednosti. Petlja `for` prolazila je kroz svaki znak ili svaki broj koji daje `range()`, a petlja `while` ponavljala se sve dok njezin uvjet nije postao lažan. U mnogim programima ipak ne želimo jednako postupiti sa svakom vrijednošću.

Pogledajmo tekst u kojem razmak samo razdvaja znakove, a `#` označava kraj korisnog dijela zapisa:

```
tekst = "A B#CD"

for znak in tekst:
    print("Obrađujem:", znak)
```

Ova petlja nema pogrešnu sintaksu i uredno će se izvršiti, ali ne poštuje značenje podataka. Razmak ne želimo obrađivati, no nakon njega trebamo nastaviti. Znak `#` također ne želimo obraditi, ali se od razmaka razlikuje po tome što nakon njega više ne trebamo pregledavati ni `C` ni `D`.

U istom programu zato mogu postojati dvije različite odluke:

1. Trenutačnu vrijednost treba zanemariti, a obradu nastaviti sa sljedećom vrijednošću
2. Cijelu petlju treba završiti jer daljnja obrada više nema smisla

Objе se odluke mogu zapisati pomoću dodatnih uvjeta i pomoćnih varijabli, ali kod postaje nepotrebno ugniježđen i teže je uočiti gdje se obrada zapravo nastavlja. Python zato ima naredbe `continue` i `break`. One ne zamjenjuju petlje ni grananja, koristimo ih unutar petlje, najčešće u naredbi `if`, kada želimo promijeniti njezin uobičajeni tijek.

8.1 Prijevremeni završetak naredbom `break`

Ponekad više nema smisla nastaviti pregled svih vrijednosti. Primjerice, poseban znak može označavati da je završio dio teksta koji trebamo obraditi.

Pretpostavimo da `#` označava kraj korisnog dijela zapisa:

```
tekst = "ABC#DEF"

for znak in tekst:
    print(znak)
```

Petlja uredno prolazi kroz tekst, ali ispisuje i znakove `D`, `E` i `F`, koje više ne želimo obrađivati. Unutar petlje možemo prepoznati `#` naredbom `if`, no treba nam naredba koja u tom trenutku zaustavlja cijelo ponavljanje. Za takav prijevremeni izlazak iz petlje koristimo naredbu `break`.

Naredba `break`

Naredba `break` odmah prekida najbližu petlju u kojoj se nalazi. Preostale naredbe te iteracije ne izvršavaju se, a program nastavlja prvom naredbom nakon petlje. `break` ne prekida cijeli program i ne odnosi se na samo grananje `if`.

Naredba `break` gotovo se uvijek nalazi unutar grananja jer petlju obično želimo prekinuti samo u određenoj situaciji. Redoslijed unutar tijela važan je: Python najprije provjerava uvjet, zatim po potrebi izvršava `break`, a naredbe zapisane iza njega u toj iteraciji više se ne dosežu.

```
for broj in range(1, 7):
    if broj == 4:
        break
    print(broj)

print("Nakon petlje.")
```

▼ Prikaži ispis i objašnjenje

```
1
2
3
Nakon petlje.
```

Za vrijednost `4` izvršava se `break` prije ispisa. Zato se ne ispisuje ni `4` ni preostale vrijednosti. Završna poruka potvrđuje da program nastavlja nakon petlje.

Tijek možemo podijeliti na tri dijela. Vrijednosti `1`, `2` i `3` ne zadovoljavaju uvjet pa se ispisuju. Vrijednost `4` zadovoljava uvjet i prekida petlju. Vrijednosti `5` i `6` nikada se ni ne dodjeljuju varijabli `broj` jer je petlja već završila. Nakon toga Python prelazi na `print("Nakon petlje.")`.

8.2 Preskakanje ostatka iteracije naredbom `continue`

Druga je situacija kada trenutačni podatak ne želimo obraditi, ali ostale i dalje trebamo pregledati. Primjerice, u tekstualnoj oznaci možemo preskočiti razmake i nastaviti s ostalim znakovima.

Bez posebne naredbe možemo cijelu obradu smjestiti u uvjet:

```
tekst = "A B"

for znak in tekst:
    if znak != " ":
        print("Znak:", znak)
        print("Veliko slovo:", znak.upper())
        print("Broj znakova:", len(znak))
```

To je ispravno, ali svaka nova naredba za prihvatljiv znak mora ostati unutar dodatnog bloka. Ako na početku iteracije već znamo da razmak ne treba obrađivati, preglednije je odmah preskočiti ostatak te iteracije i nastaviti sa sljedećim znakom.

💡 Naredba `continue`

Naredba `continue` preskače preostale naredbe trenutačne iteracije. U petlji `for` slijedi preuzimanje sljedeće vrijednosti, a u petlji `while` nova provjera uvjeta. Petlja se time ne završava.

Za razliku od `break`, `continue` ne napušta petlju. Možemo ga čitati kao „s ovim podatkom više ništa ne radi, prijede na sljedeću iteraciju”. Najčešće ga koristimo nakon provjere podatka koji treba zanemariti, dok se glavni dio obrade nalazi nakon te provjere.

```
for broj in range(1, 7):
    if broj == 4:
        continue
    print(broj)

print("Nakon petlje.")
```

▼ Prikaži ispis i usporedbu

```
1
2
3
5
6
Nakon petlje.
```

Za razliku od prethodnog primjera, preskočen je samo ispis broja `4`. Vrijednosti `5` i `6` obrađene su u sljedećim iteracijama.

Kada je `broj` jednak `4`, Python izvršava `continue` i vraća se na početak petlje po sljedeću vrijednost. Poziv `print(broj)` u toj se iteraciji preskače. Za `5` se uvjet ponovno provjerava, daje `False` i ispis se normalno izvršava.

Naredba za <code>broj == 4</code>	Posljedica
<code>break</code>	Završava petlju, više se ne obrađuju <code>5</code> ni <code>6</code>
<code>continue</code>	Preskače ostatak te iteracije, nastavlja na ostatak petlje

Naredbe `break` i `continue` nisu obavezne u svakoj petlji. Primjer s preskakanjem četvorke mogao bi se zapisati i uvjetnim ispisom `if broj != 4:`. Koristimo ih kada olakšavaju praćenje postupka.

8.3 while True i završna oznaka

Kod niza korisničkih unosa često unaprijed ne znamo koliko će vrijednosti korisnik unijeti. Korisnik zato može posebnom riječju označiti da više nema podataka. Takvu vrijednost zovemo završnom oznakom: ona upravlja obradom, ali nije podatak koji treba uračunati.

Postupak možemo napisati običnom petljom `while`, ali tada isti unos moramo zatražiti prije petlje i ponovno na kraju njezina tijela:

```
naziv = input("Naziv proizvoda ili kraj: ").strip()

while naziv.lower() != "kraj":
    print("Prihvaćen proizvod:", naziv)
    naziv = input("Naziv proizvoda ili kraj: ").strip()
```

Ovakav je program ispravan, ali naredba za unos ponavlja se na dva mjesta. Ako kasnije dodamo provjeru praznog unosa ili još neko pravilo, moramo paziti da svaki put ponovno zatražimo novu vrijednost. U suprotnom petlja može ostati na istom unosu ili se pojedini slučajevi mogu obrađivati različito.

Druga je mogućnost postaviti petlju koja se sama ne zaustavlja, unos tražiti samo na jednom mjestu, a završetak provjeriti odmah nakon unosa. Kada program prepozna završnu oznaku, naredba `break` prekida petlju.

Zapis `while True:` namjerno postavlja uvjet koji je uvijek istinit. Takva petlja nema završetak kroz promjenu uvjeta, pa u njezinu tijelu mora postojati dosežna odluka koja izvršava `break`. Prednost ovog oblika jest što novi unos možemo odmah provjeriti i završiti na mjestu na kojem prepoznamo završnu oznaku.

Završna oznaka, često nazvana i sentinel, posebna je vrijednost koja nije dio podataka. Ako korisnik upiše `kraj`, taj tekst ne želimo brojiti kao naziv proizvoda. Zato se provjera završetka nalazi prije povećavanja brojača i prije ostale obrade.

```
broj_unosa = 0

while True:
    naziv = input("Naziv proizvoda ili kraj: ").strip()
    if naziv.lower() == "kraj":
        break
    if naziv == "":
        print("Naziv ne smije biti prazan.")
        continue

    broj_unosa += 1
    print("Prihvaćen proizvod:", naziv)

print("Broj prihvaćenih proizvoda:", broj_unosa)
```

▼ Prikaži primjer unosa i ispisa

```
Naziv proizvoda ili kraj: Bilježnica
Prihvaćen proizvod: Bilježnica
Naziv proizvoda ili kraj:
Naziv ne smije biti prazan.
Naziv proizvoda ili kraj: Olovka
Prihvaćen proizvod: Olovka
Naziv proizvoda ili kraj: KRAJ
Broj prihvaćenih proizvoda: 2
```

Provjera završne oznake dolazi prije brojanja. Prazan unos dovodi do `continue`, pa se također ne broji. Uvjet `True` sam nikada neće postati lažan, ovaj program završava petlju kada se izvrši `break`.

Redoslijed provjera opisuje pravila programa:

1. Ako je unesena završna oznaka, petlja se prekida.
2. Ako je unos prazan, prikazuje se poruka i odmah traži novi unos.
3. Svi ostali unosi povećavaju brojač i obrađuju se kao proizvodi.

Premještanje brojanja iznad tih provjera pogrešno bi uključilo završnu oznaku ili prazan tekst u rezultat.

8.4 Continue i promjena brojača u petlji while

U petlji `for` Python na početku svake iteracije automatski dodjeljuje sljedeću vrijednost iz izvora varijabli petlje. Primjerice, u zapisu `for broj in range(5):` varijabla `broj` redom dobiva vrijednosti od `0` do `4`, pa ih ne moramo sami mijenjati. U petlji `while` nema takve automatske promjene: sami moramo promijeniti vrijednost o kojoj ovisi uvjet petlje. Ako `continue` preskoči upravo tu promjenu, petlja može ponovno provjeravati istu vrijednost i tako se ponavljati unedogled.

U sljedećem programu brojač se povećava na kraju tijela. Kada `broj` postane `3`, naredba `continue` preskače povećavanje. Vrijednost zato ostaje `3`, isti uvjet ponovno je istinit i petlja se više ne može završiti:

```
broj = 0

while broj < 5:
    if broj == 3:
        continue
    print(broj)
    broj += 1
```

Ovaj primjer nemojte pokretati bez zaustavljanja programa jer bi se petlja ponavljala unedogled. Problem nije u samoj naredbi `continue`, nego u tome što svaki put preskače jedinu naredbu koja bi promijenila vrijednost korištenu u uvjetu petlje.

Zato pri svakoj naredbi `continue` u petlji `while` treba pratiti koje se naredbe nalaze između nje i kraja tijela. Ako se ondje nalazi jedina promjena varijable iz uvjeta, ta se promjena u preskočenom slučaju neće dogoditi. Problem se može riješiti promjenom stanja prije provjere ili drukčijom organizacijom uvjeta.

Provjerite sve putove kroz petlju

Ako povećavanje brojača stoji iza `continue`, u toj se iteraciji neće izvršiti. Provjerite napreduje li program i u slučaju koji se preskače. To je osobito važno kada brojač sudjeluje u uvjetu petlje `while`.

Sljedeći ispravan primjer povećava brojač prije mogućeg preskakanja:

```
broj = 0

while broj < 5:
    broj += 1
    if broj == 3:
        continue
    print(broj)
```

▼ Prikaži očekivani ispis

1
2
4
5

Brojač napreduje i kada se ispis preskoči. Nakon iteracije s vrijednošću 5 uvjet postaje lažan.

Pregled novih naredbi

Naredba	Namjena
<code>break</code>	Prekida najbližu petlju i nastavlja nakon nje
<code>continue</code>	Preskače ostatak trenutne iteracije

Zadatak

U tekstu "A B C#D E" znak # označava kraj dijela koji treba obraditi. Ispišite svaki znak prije te oznake u zasebnom retku, ali preskočite razmake. Nakon obrade jednom ispišite poruku `Obrada završena`. Znak # i sve iza njega ne smiju se ispisati.

▼ Prikaži moguće rješenje i ispis

```
tekst = "A B C#D E"

for znak in tekst:
    if znak == "#":
        break
    if znak == " ":
        continue
    print(znak)

print("Obrada završena")
```

A
B
C
Obrada završena

Ako # ne postoji, pregledat će se cijeli tekst. Ako je # prvi znak, ispisat će se samo završna poruka.

9 Ugniježdene petlje

9.1 Ponavljanje postupka unutar drugog ponavljanja

Zamislite dvoranu koja ima dva reda, a u svakom se redu nalaze tri mjesta. Svako mjesto želimo označiti brojem reda i brojem mjesta unutar tog reda. Program zato treba ispisati šest oznaka: 1-1, 1-2, 1-3, 2-1, 2-2 i 2-3.

Jedna vrijednost nije dovoljna za određivanje mjesta. Za svaku oznaku moramo povezati dvije vrijednosti: trenutni red i trenutno mjesto u tom redu. Najprije obrađujemo prvi red i njegova tri mjesta, a zatim drugi red i ponovno mjesta od jedan do tri.

Budući da redova zasad ima samo dva, mogli bismo napisati jednu petlju za mjesta u prvom redu i zatim gotovo istu petlju za mjesta u drugom redu:

```
for mjesto in range(1, 4):
    print(f"Mjesto 1-{mjesto}")

for mjesto in range(1, 4):
    print(f"Mjesto 2-{mjesto}")
```

Prva petlja ispisuje oznake 1-1, 1-2 i 1-3, a druga 2-1, 2-2 i 2-3. Program daje traženi rezultat, ali se isti postupak zapisuje dvaput. Za svaki novi red morali bismo dodati još jednu gotovo jednaku petlju i u njoj ručno promijeniti broj reda.

Brojevi redova također čine pravilno ponavljanje: najprije trebamo red 1, a zatim red 2. Zato jednu petlju možemo koristiti za prolazak kroz redove. Unutar svake njezine iteracije druga petlja prolazi kroz sva tri mjesta trenutnog reda. Tako za svaki red dobivamo sve pripadajuće kombinacije reda i mjesta.

Ugniježdene petlje

Ugniježdene petlje nastaju kada se jedna petlja nalazi u tijelu druge. Za svaku iteraciju vanjske petlje unutarnja petlja ponovno započinje i izvršava svoje iteracije. Vanjska petlja nastavlja sljedećom vrijednošću tek nakon završetka unutarnje.

Vanjska petlja određuje širu cjelinu, u ovom primjeru red. Unutarnja petlja obrađuje sve elemente unutar te cjeline, odnosno mjesta. Budući da je unutarnja petlja uvučena unutar vanjske, ona se izvršava od početka za svaku vrijednost varijable `red`.

Redoslijed nije `red 1, red 2, pa sva mjesta`. Python najprije uzima `red = 1`, prolazi sva tri mjesta, zatim uzima `red = 2` i ponovno prolazi sva tri mjesta. Jednu potpunu unutarnju petlju možemo promatrati kao dio jedne iteracije vanjske petlje.

```
for red in range(1, 3):
    print("Red:", red)
    for mjesto in range(1, 4):
        print(f"Mjesto {red}-{mjesto}")

print("Sva mjesta su ispisana.")
```

▼ Prikaži ispis i redosljed vrijednosti

Red: 1

Mjesto 1-1

Mjesto 1-2

Mjesto 1-3

Red: 2

Mjesto 2-1

Mjesto 2-2

Mjesto 2-3

Sva mjesta su ispisana.

Redosljed ispisa mjesta	red	mjesto
1	1	1
2	1	2
3	1	3
4	2	1
5	2	2
6	2	3

Naredba za ispis mjesta izvršava se $2 * 3$, odnosno šest puta. Naredba za ispis reda izvršava se dvaput, a završna poruka jednom. Broj ponavljanja uvijek ovisi o tome kojem bloku naredba pripada.

Ako vanjska petlja ima a iteracija, a unutarnja za svaku od njih b iteracija, tijelo unutarnje petlje izvršava se $a * b$ puta. To vrijedi kada unutarnja petlja svaki put prolazi isti broj vrijednosti i nema ranog prekida. U primjeru su to dva reda puta tri mjesta.

9.2 Ponovno postavljanje unutarnjeg brojača

Isti postupak možemo zapisati petljama `while`. Tada moramo sami postaviti i povećavati oba brojača. Posebno je važno početnu vrijednost mjesta postaviti svaki put kada prijeđemo na novi red.

Varijabla `red` pripada vanjskom ponavljanju i postavlja se prije vanjske petlje. Varijabla `mjesto` pripada jednom prolasku kroz trenutni red. Zato se postavlja unutar vanjske, ali prije unutarnje petlje. Taj položaj znači da svaki novi red ponovno počinje od mjesta `1`.

```
red = 1

while red <= 2:
    mjesto = 1
    while mjesto <= 3:
        print(f"Mjesto {red}-{mjesto}")
        mjesto += 1
    red += 1
```

▼ Prikaži ispis i objašnjenje

Mjesto 1-1
Mjesto 1-2
Mjesto 1-3
Mjesto 2-1
Mjesto 2-2
Mjesto 2-3

Nakon prvog reda varijabla `mjesto` ima vrijednost `4`. Zato je prije obrade drugog reda ponovno postavljamo na `1`. Kada bi početna dodjela bila samo prije vanjske petlje, unutarnja se petlja za drugi red više ne bi izvršila.

Na kraju prvog prolaska `red` još uvijek iznosi `1`, a `mjesto` je `4`. Unutarnji uvjet tada postaje lažan, povećava se `red` na `2` i započinje nova iteracija vanjske petlje. Naredba `mjesto = 1` ponovno se izvršava prije nove unutarnje provjere. Bez nje bi uvjet `mjesto <= 3` odmah bio lažan.

9.3 Doseg naredbe `break` u ugniježđenim petljama

Naredba `break` prekida samo najbližu petlju. Ako se nalazi u unutarnjoj petlji za mjesta, vanjska petlja za redove nastavlja raditi. To je važno kada očekujemo da jedan pronalazak prekine cijeli postupak.

Python ne pretpostavlja da želimo napustiti sve razine ponavljanja. Naredba se odnosi na petlju čijem tijelu izravno pripada. Nakon prekida unutarnje petlje izvršavanje se nastavlja prvom naredbom koja slijedi iza nje, a ona se još uvijek može nalaziti u tijelu vanjske petlje.

```
for red in range(1, 3):
    for mjesto in range(1, 4):
        if mjesto == 2:
            break
        print(f"Mjesto {red}-{mjesto}")
```

▼ Prikaži očekivani ispis

Mjesto 1-1
Mjesto 2-1

U svakom redu unutarnja petlja završava pri vrijednosti `2`. Vanjska ipak prelazi na sljedeći red.

Za `red = 1` ispisuje se samo mjesto `1-1`, zatim unutarnji `break` prekida mjesta tog reda. Vanjska petlja potom postavlja `red = 2`, stvara novu unutarnju petlju i ispisuje `2-1`. Ako bismo željeli završiti obje petlje, trebali bismo dodatno prenijeti informaciju vanjskoj petlji, takav postupak zasad nam nije potreban.

Zadatak

Ormari su označeni slovima od `A` do `I`, a svaki ima police označene brojevima od `1` do `5`. Ormar `D` trenutno nije dostupan pa ga treba potpuno preskočiti. Ormari od `G` nadalje pripadaju drugom odjelu, kada obrada dođe do ormara `G`, cijeli pregled ormara treba završiti. U svakom dostupnom ormaru oprema se nalazi samo na policama `1`, `2` i `3`. Kada pregled dođe do police `4`, pregled polica tog ormara treba završiti i nastaviti sa sljedećim ormarom.

Ispišite oznaku svake pregledane police, primjerice `A-1`, u zasebnom retku. Nakon pregledanih polica svakog obrađenog ormara ispišite da je njegov popis završen. Na kraju jednom ispišite ukupan broj pregledanih polica.

▼ Prikaži moguće rješenje i ispis

```
broj_polica = 0

for ormar in "ABCDEFGHI":
    if ormar == "G":
        break
    if ormar == "D":
        continue

    for polica in range(1, 6):
        if polica == 4:
            break
        print(f"{ormar}-{polica}")
        broj_polica += 1
    print(f"Popis ormara {ormar} završen.")

print("Ukupno polica:", broj_polica)
```

```
A-1
A-2
A-3
Popis ormara A završen.
B-1
B-2
B-3
Popis ormara B završen.
C-1
C-2
C-3
Popis ormara C završen.
E-1
E-2
E-3
Popis ormara E završen.
F-1
F-2
F-3
Popis ormara F završen.
Ukupno polica: 15
```

Kada je `ormar` jednak `D`, naredba `continue` preskače ostatak te iteracije vanjske petlje, pa se unutarnja petlja za njegove police uopće ne pokreće. Za ormare `A`, `B`, `C`, `E` i `F` unutarnja petlja ispisuje prve tri police. Vrijednost `4` izvršava unutarnji `break`, koji završava samo pregled polica trenutnog ormara, pa vanjska petlja nastavlja sa sljedećim ormarom. Kada vanjska petlja dođe do oznake `G`, njezin `break` završava cijeli pregled i oznake `G`, `H` i `I` ne ispisuju se.

10 Primjer: obrada više unesenih rezultata

10.1 Pravila obrade i priprema postupka

Sada ćemo povezati unos podataka, provjeru valjanosti, ponavljanje i izradu izvještaja. Program treba obrađivati proizvoljan broj rezultata bez potrebe da za svakog studenta dodajemo novi redak koda. Nećemo pohranjivati pojedinačne rezultate, nego podatke potrebne za završni izvještaj.

Takav program ima dvije vremenski odvojene cjeline. Tijekom unosa stalno prima i provjerava pojedinačne vrijednosti te ažurira rezultate. Nakon završne oznake prestaje primati podatke i izrađuje izvještaj. Naredbe završnog izvještaja zato se nalaze izvan petlje.

Zadatak: Korisnik unosi cjelobrojne bodove od 0 do 100. Riječ `kraj`, neovisno o veličini slova i rubnim razmacima, završava unos. Prazan unos, tekst koji nije sastavljen od ASCII znamenki 0–9 i broj izvan dopuštenog raspona treba odbiti uz odgovarajuću poruku. Za prihvaćene rezultate izračunajte njihov broj, zbroj, prosjek i broj rezultata s najmanje 60 bodova. Ako nema prihvaćenih rezultata, umjesto prosjeka ispišite objašnjenje.

Ovdje namjerno prihvaćamo samo nenegativne cijele brojeve zapisane običnim znamenkama, s najviše tri znamenke u zapisu. Unosi poput `-5`, `72.5`, `+60` i `0000` ne odgovaraju tom formatu. To je pravilo ovog zadatka, a ne tvrdnja da ih Python inače ne može tumačiti kao brojeve.

Prije pisanja petlje treba odrediti što program mora pamtit i između dvaju unosa. Za završni izvještaj potrebne su ove varijable:

Varijabla	Početna vrijednost	Što predstavlja tijekom obrade?
<code>broj_prihvacenih</code>	0	Koliko je valjanih rezultata dosad obrađeno
<code>broj_iznad_praga</code>	0	Koliko prihvaćenih rezultata ima najmanje 60 bodova
<code>zbroj</code>	0	Zbroj bodova svih prihvaćenih rezultata

Zasebne rezultate nije potrebno čuvati jer zadatak ne traži njihov kasniji pojedinačni ispis. Za prosjek su dovoljni konačan zbroj i broj prihvaćenih vrijednosti. Ako bi zadatak poslije tražio medijan ili ponovni pregled svakog rezultata, trebala bi nam struktura koja čuva pojedinačne podatke, takve ćemo strukture obraditi naknadno.

Postupak:

1. Postavite brojače i zbroj na nulu.
2. Zatražite novi unos i uklonite rubne razmake.
3. Provjerite je li korisnik zatražio završetak.
4. Provjerite format zapisa prije pretvorbe u cijeli broj.
5. Pretvorite prihvatljiv zapis i provjerite brojčani raspon.
6. Ažurirajte zbroj i brojače samo za prihvaćen rezultat.

7. Nakon završetka unosa ispišite izvještaj i izračunajte prosjek samo ako postoji barem jedan prihvaćen rezultat.

Redoslijed ovih koraka nije proizvoljan. Završnu oznaku moramo prepoznati prije pretvorbe jer `int("kraj")` nije dopušten. Zapis provjeravamo prije poziva `int()` kako nebrojčani unos ne bi zaustavio program. Brojače ažuriramo tek nakon svih provjera kako odbačeni pokušaji ne bi utjecali na statistiku.

10.2 Program i završni izvještaj

Prije otvaranja rješenja pokušajte odrediti koje naredbe pripadaju petlji, a koje moraju stajati nakon nje. Posebno razmislite gdje treba provjeriti završnu oznaku: riječ `kraj` ne može se pretvoriti u cijeli broj.

Unutar jedne iteracije moguća su četiri ishoda:

Vrsta unosa	Radnja programa
Završna oznaka	Izvršava <code>break</code> i prelazi na izvještaj
Neispravan zapis	Ispisuje poruku, izvršava <code>continue</code> i traži novi unos
Broj izvan raspona	Ispisuje poruku, izvršava <code>continue</code> i traži novi unos
Prihvatljiv rezultat	Ažurira zbroj i brojače te nastavlja sljedećom iteracijom

Samo posljednji put vodi do promjene podataka završnog izvještaja.

▼ Prikaži kod, primjer unosa i objašnjenje

```
broj_prihvacenih = 0
broj_iznad_praga = 0
zbroj = 0

while True:
    unos = input("Bodovi od 0 do 100 ili kraj: ").strip()

    if unos.lower() == "kraj":
        break

    zapis_ispravan = unos != ""
    for znak in unos:
        if znak not in "0123456789":
            zapis_ispravan = False
            break

    if not zapis_ispravan:
        print("Upišite cijeli broj znamenkama od 0 do 9.")
        continue

    if len(unos) > 3:
        print("Upišite najviše tri znamenke.")
        continue

    bodovi = int(unos)
    if bodovi > 100:
        print("Bodovi moraju biti od 0 do 100.")
        continue

    broj_prihvacenih += 1
    zbroj += bodovi
    if bodovi >= 60:
        broj_iznad_praga += 1
    print("Rezultat je prihvaćen.")

print("Prihvaćeno rezultata:", broj_prihvacenih)
print("Zbroj bodova:", zbroj)
print("Rezultata s najmanje 60 bodova:", broj_iznad_praga)

if broj_prihvacenih > 0:
    prosjek = zbroj / broj_prihvacenih
    print(f"Prosjek: {prosjek:.2f}")
else:
    print("Prosjek nije moguće izračunati jer nema prihvaćenih rezultata.")
```

```
Bodovi od 0 do 100 ili kraj: 72
Rezultat je prihvaćen.
Bodovi od 0 do 100 ili kraj: 120
Bodovi moraju biti od 0 do 100.
Bodovi od 0 do 100 ili kraj: abc
Upišite cijeli broj znamenkama od 0 do 9.
Bodovi od 0 do 100 ili kraj: 48
Rezultat je prihvaćen.
Bodovi od 0 do 100 ili kraj: 90
Rezultat je prihvaćen.
Bodovi od 0 do 100 ili kraj: kraj
Prihvaćeno rezultata: 3
Zbroj bodova: 210
Rezultata s najmanje 60 bodova: 2
Prosjek: 70.00
```

Varijabla `zapis_ispravan` pamti rezultat provjere znakova. Prazan unos odmah daje `False`, inače svaki znak provjeravamo prema dopuštenim znamenkama. `break` u toj provjeri prekida samo unutarnju petlju `for`. Vanjska petlja `while` nastavlja do provjere dobivenog rezultata.

Za unos `"72"` varijabla `zapis_ispravan` počinje vrijednošću `True`. Unutarnja petlja zasebno provjerava znak `"7"` i znak `"2"`, oba se nalaze u tekstu `"0123456789"`, pa vrijednost ostaje `True`. Duljina je dopuštena, pretvorba daje broj `72`, a broj pripada rasponu. Tek tada se povećavaju brojači i zbroj.

Za unos `"abc"` prvi znak `"a"` nije dopuštena znamenka. Program postavlja `zapis_ispravan = False` i prekida unutarnju petlju jer daljnji znakovi više ne mogu promijeniti zaključak. Vanjski `if not zapis_ispravan:` ispisuje poruku, a `continue` preskače pretvorbu i sva ažuriranja rezultata.

Provjera duljine ograničava zapis na najviše tri znamenke. To je dovoljno za zadani raspon i izbjegava pretvaranje proizvoljno dugog unosa. Nakon provjere znakova znamo da broj nije negativan, pa pri provjeri raspona ostaje samo gornja granica.

Zapis od tri znamenke još uvijek može predstavljati nedopušten broj, primjerice `120`, pa provjera duljine ne zamjenjuje provjeru raspona. S druge strane, tekst `"000"` prolazi zadano pravilo zapisa i pretvara se u broj `0`. Početne nule u ovom zadatku nisu važne jer obrađujemo količinu bodova, a ne identifikacijsku oznaku.

Svaki `continue` u vanjskoj petlji vraća program na novi unos prije zbrajanja i brojanja. Zato se odbačeni podaci ne nalaze u završnom izvještaju. Nula je valjan rezultat: povećava broj prihvaćenih, iako ne povećava zbroj.

Prosjek se računa nakon petlje jer tada znamo konačan zbroj i konačan broj prihvaćenih rezultata. Provjera `broj_prihvacenih > 0` sprječava dijeljenje nulom. Kada nema prihvaćenih podataka, brojčani prosjek ne postoji i smislenije je ispisati objašnjenje nego izmisliti vrijednost nula.

11 Pogreške i provjera programa

11.1 Sintaksa i pripadnost bloku

Kod grananja i petlji često se pojavljuju pogreške koje ranije nismo susretali: nedostaje dvotočka, tijelo nije uvučeno ili naredbe nisu poravnate. Python neke od njih prepoznaje kao sintaksne pogreške. Druge su dopuštene, ali mijenjaju značenje programa.

Sintaksna pogreška znači da Python ne može pravilno protumačiti zapis programa. Izvršavanje se tada zaustavlja i dobivamo poruku pogreške. Logička pogreška znači da je kod dopušten i može se izvršiti, ali ne opisuje postupak koji smo namjeravali. Primjerice, završni ispis unutar petlje nije zabranjen, no daje više ispisa umjesto jednoga.

Problem	Posljedica
Nedostaje <code>:</code> nakon uvjeta ili zaglavlja petlje	<code>SyntaxError</code>
Nakon zaglavlja nema pravilno uvučenog tijela	Obično <code>IndentationError</code>
Nedosljedno miješanje tabulatora i razmaka	Može izazvati <code>TabError</code>
Ispis završnog rezultata ostao je unutar petlje	Više ispisa međurezultata umjesto jednog završnog ispisa
Varijabla se dodjeljuje samo u grani koja nije izvršena	Kasnije korištenje može izazvati <code>NameError</code>

Poruke pogrešaka mogu se ponešto razlikovati među verzijama Pythona. Pročitajte vrstu pogreške i označeni redak, a zatim pregledajte i redak neposredno prije njega.

Kod pogreške uvlačenja nemojte nasumično dodavati razmake samo na označenom retku. Najprije pronađite zaglavlje `if`, `for` ili `while`, a zatim odredite koje mu naredbe sadržajno pripadaju. Naredbe istog bloka poravnajte jednako. Ugniježđeni blok treba biti uvučen još jednu razinu u odnosu na strukturu u kojoj se nalazi.

11.2 Logičke pogreške pri ponavljanju

Sljedeći program želi izračunati zbroj od `1` do uključivo `4`, ali ima dvije pogreške. Prije otvaranja objašnjenja pratite koju vrijednost `zbroj` ima nakon svake iteracije.

```
for broj in range(1, 4):
    zbroj = 0
    zbroj += broj

print(zbroj)
```

▼ Prikaži ispis, objašnjenje i ispravak

Program ispisuje `3`. Raspon ne uključuje `4`, a zbroj se u svakoj iteraciji ponovno postavlja na nulu. Potrebna su dva odvojena ispravka:

```
zbroj = 0

for broj in range(1, 5):
    zbroj += broj

print(zbroj)
```

10

11.3 Sustavna provjera izvršavanja

Kada se program izvrši, ali daje pogrešan rezultat, privremeni ispisi mogu pokazati gdje se očekivanje razilazi sa stvarnim stanjem. Korisno je ispisati vrijednost prije provjere ili prije i nakon promjene, uz jasne oznake. Kod kratkih primjera isto možemo pratiti tablicom.

Primjerice, za pogrešan zbroj možemo privremeno dodati `print("broj:", broj, "zbroj:", zbroj)` unutar petlje. Ako se zbroj u svakoj iteraciji vraća na nulu, odmah ćemo vidjeti da je početna dodjela na pogrešnom mjestu. Privremeni ispis ne popravljia program, on samo čini nevidljivo stanje varijabli vidljivim kako bismo pronašli uzrok.

Postupak:

1. Zapišite što program treba napraviti za mali, lako provjerljiv primjer
2. Odredite početne vrijednosti svih brojača i akumulatora
3. Pratite uvjet i promjene vrijednosti kroz nekoliko iteracija
4. Provjerite pripadnost svake naredbe bloku
5. Provjerite slučaj bez iteracija, jednu iteraciju i više iteracija
6. Provjerite točne granične vrijednosti i vrijednosti neposredno uz njih
7. Nakon ispravka uklonite privremene ispise koji nisu dio traženog rezultata.

Za petlju `while` dodatno pratite može li se stanje promijeniti tako da uvjet postane lažan ili da se izvrši `break`. Ako se ponavlja ista vrijednost, najprije provjerite mjesto promjene brojača i djelovanje naredbe `continue`.

Ako zapnete pri samostalnom radu, javite se putem Google Chata ili e-maila. Priložite datoteku s kodom, cijelu poruku pogreške ako postoji, vrijednosti koje ste unosili te kratak opis očekivanog rezultata.

Zadatak

Program treba unijeti tri iznosa, zbrojiti ih i jednom ispisati ukupan iznos. Za unose 5, 10 i 15 očekuje se rezultat 30.00. Zašto sljedeći program ne ispunjava taj zahtjev i što treba promijeniti?

```
ukupno = 0

for redni_broj in range(1, 3):
    iznos = float(input("Iznos: "))
    ukupno = iznos
    print(f"Ukupno: {ukupno:.2f}")
```

▼ Prikaži moguće rješenje i objašnjenje

```
ukupno = 0

for redni_broj in range(1, 4):
    iznos = float(input("Iznos: "))
    ukupno += iznos

print(f"Ukupno: {ukupno:.2f}")
```

```
Iznos: 5
Iznos: 10
Iznos: 15
Ukupno: 30.00
```

Treba uključiti treću iteraciju, dodavati iznos postojećem zbroju umjesto zamjenjivati ga te završni ispis premjestiti izvan petlje. Pretpostavljamo ispravan brožčani unos.

12 Pregled naredbi, funkcija i metoda

Pregled okuplja elemente korištene u ovoj skripti. Ključne riječi i upravljačke strukture odvojene su od funkcija i metoda: primjerice, `if` nije funkcija i ne poziva se kao `print()`.

12.1 Grananja i petlje

Zapis	Namjena
<code>if uvjet:</code>	Uvjetno izvršavanje bloka
<code>elif uvjet:</code>	Sljedeća mogućnost u povezanom grananju
<code>else:</code>	Alternativna grana kada prethodni uvjeti nisu zadovoljeni
<code>for vrijednost in izvor:</code>	Obrada vrijednosti iz izvora, jedne po jedne
<code>while uvjet:</code>	Ponavljanje dok je uvjet istinit
<code>break</code>	Prijevreteni izlazak iz najbliže petlje
<code>continue</code>	Preskakanje ostatka iteracije

12.2 Funkcije

Funkcija	Namjena
<code>float()</code>	Pretvara prikladan unos u decimalni broj
<code>input()</code>	Prikazuje upit i vraća korisnički unos kao tekst
<code>int()</code>	Pretvara prikladan unos u cijeli broj
<code>len()</code>	Vraća duljinu teksta
<code>print()</code>	Ispisuje vrijednosti i poruke
<code>range()</code>	Predstavlja niz cijelih brojeva zadan početkom, krajem i korakom

12.3 Metode tipa str

Metoda	Namjena
<code>count()</code>	Broji pojavljivanja dijela teksta, spomenuta kao alternativa ručnom brojanju
<code>lower()</code>	Vraća tekst s malim slovima
<code>strip()</code>	Vraća tekst bez rubnih bjelina
<code>upper()</code>	Vraća tekst s velikim slovima

13 Samostalni zadaci

Svaki zadatak spremite u zasebnu Python datoteku, primjerice `zadatak_01.py`. Koristite opisna imena varijabli i jasno označene ispise. Rješenja na kraju skripte otvorite nakon vlastitog pokušaja. Mogući su različiti ispravni postupci, važno je da program ispunjava sva pravila zadatka i radi za više ulaznih vrijednosti.

U zadacima s broječanim unosom možete pretpostaviti da korisnik unosi valjan zapis traženog broja s decimalnom točkom, osim kada se izričito traži provjera tekstualnog zapisa. Zadane raspone i ostale uvjete ipak treba provjeravati prema opisu zadatka.

13.1 Zadatak 1

Temperatura rashladnog prostora iznosi 9 stupnjeva. Dopuštena temperatura je od 2 do 8 stupnjeva, uključujući granice. Napišite program koji razlikuje prenisku, dopuštenu i previsoku temperaturu te ispisuje odgovarajuću poruku. Nakon poruke uvijek ispišite izmjerenu temperaturu. Provjerite program i za obje granice dopuštenog raspona.

13.2 Zadatak 2

Za posudbu uređaja korisnički račun mora biti aktivan. Ako nije aktivan, treba ispisati poruku o tome. Ako je aktivan, posudba je moguća samo ako korisnik nema nepodmirenih obveza, u suprotnom treba ispisati da prvo mora podmiriti obveze. Početne vrijednosti neka budu `racun_aktivan = True` i `nepodmirene_obveze = True`. Napišite program koji ispisuje jednu odgovarajuću poruku i provjerite sve slučajeve opisane u zadatku.

13.3 Zadatak 3

Broj osvojenih bodova mora biti cijeli broj od 0 do 100. Korisnik unosi broj, a program najprije treba odbiti vrijednost izvan dopuštenog raspona. Valjan rezultat manji od 50 označite kao `Potrebno ponavljanje`, rezultat od 50 do manje od 80 kao `Zadovoljava`, a rezultat od 80 nadalje kao `Vrlo uspješno`. Ispišite jednu poruku koja odgovara unesenoj vrijednosti.

13.4 Zadatak 4

Laboratorijski uzorci označavaju se brojevima od 12 do 20, uključujući oba kraja. Ispišite oznaku svakog uzorka u obliku `Uzorak 12`, svaku u zasebnom retku. Nakon svih oznaka jednom ispišite poruku da je označavanje završeno.

13.5 Zadatak 5

Sustav treba pripremiti podsjetnike za preostalo vrijeme od 30, 25, 20, 15, 10 i 5 minuta. Za svaku vrijednost ispišite poruku koliko je minuta preostalo. Nakon posljednjeg podsjetnika jednom ispišite `Vrijeme je isteklo`.

13.6 Zadatak 6

Korisnik treba unijeti četiri nenegativna decimalna iznosa troškova. Nakon sva četiri unosa ispišite ukupan trošak i prosječan trošak, oba s dvije decimale. Zasebno ispišite koliko je unesenih troškova strogo veće od 20 eura. Pretpostavite da korisnik poštuje uvjet nenegativnosti.

13.7 Zadatak 7

Za cijele brojeve od 10 do 35, uključujući granice, pronađite one koji su djeljivi s 4. Ispišite svaku pronađenu vrijednost, a nakon obrade cijelog raspona njihov broj i zbroj. Svaki od završnih rezultata treba imati jasnu oznaku.

13.8 Zadatak 8

Zapis senzora glasi "A2B40C7". Odredite koliko njegovih znakova pripada znamenkama od 0 do 9 te izračunajte zbroj tih znamenki. Znakove 4 i 0 treba obraditi zasebno, a ne kao broj 40. Ispišite broj znamenki i njihov zbroj.

13.9 Zadatak 9

Za obradu jedne kutije potrebno je 7 minuta, a do završetka smjene ostale su 32 minute. Pripremajte kutije dok ima dovoljno vremena za cijelu sljedeću kutiju. Nakon svake obrade ispišite broj obrađenih kutija i preostalo vrijeme. Na kraju ispišite koliko je kutija ukupno obrađeno i koliko je minuta ostalo neiskorišteno.

13.10 Zadatak 10

Program od korisnika traži oznaku veličine paketa. Prihvatljive su oznake S, M i L, neovisno o veličini slova i razmacima na početku ili kraju unosa. Ako unos nije prihvatljiv, ispišite objašnjenje i zatražite novu oznaku. Nakon prihvaćenog unosa ispišite odabranu veličinu velikim slovom.

13.11 Zadatak 11

Korisnik unosi cijele bodove od 0 do 10, a broj -1 označava završetak. Ostale brojeve izvan dopuštenog raspona odbijte uz poruku i nastavite tražiti novi unos. Na kraju ispišite broj prihvaćenih rezultata i njihov prosjek s dvije decimale. Ako nije prihvaćen nijedan rezultat, ispišite poruku da prosjek nije dostupan. Završna oznaka ne ulazi u rezultate.

13.12 Zadatak 12

U zapisu "ab-3-c!de-9" znak ! označava kraj dijela koji treba obraditi. Izgradite novi tekst od znakova prije te oznake, izostavljajući crtice. Nakon obrade ispišite dobiveni tekst i broj zadržanih znakova. Provjerite ponašanje i kada završna oznaka ne postoji ili se nalazi na samom početku.

13.13 Zadatak 13

U spremištu su ormari **X** i **Y**, a svaki ima pretince od **1** do **3**. Ispišite sve oznake pretinaca u obliku **X1**. Pretinac **Y2** nije dostupan, pa uz njegovu oznaku treba stajati riječ **nedostupan**, a uz sve ostale **dostupan**. Na kraju ispišite ukupan broj dostupnih pretinaca.

13.14 Zadatak 14

Za svaki cijeli broj od **1** do **20** ispišite jednu poruku. Ako je broj djeljiv i s **2** i s **3**, ispišite **Oboje**. Ako je djeljiv samo s **2**, ispišite **Dva**, a ako je djeljiv samo s **3**, ispišite **Tri**. Za ostale vrijednosti ispišite sam broj. Redoslijed ispisa mora pratiti rastući redoslijed obrađenih brojeva.

13.15 Zadatak 15

Sljedeći program treba obraditi brojeve od **1** do **5**, izostaviti broj **3** te jednom ispisati zbroj ostalih vrijednosti. Trenutačno se ne zaustavlja. Najprije čitanjem koda utvrdite zašto, a zatim ga ispravite tako da daje traženi rezultat. Ako ga pokrenete prije ispravka, prekinite izvršavanje u terminalu.

```
broj = 1
zbroj = 0

while broj <= 5:
    if broj == 3:
        continue
    zbroj += broj
    broj += 1

print(zbroj)
```

14 Rješenja samostalnih zadataka

Prikazana rješenja jedan su od mogućih načina. Usporedite i postupak i rezultat. Kod zadataka s unosom ponovite provjeru s drugim vrijednostima, osobito graničnima.

14.1 Rješenje zadatka 1

▼ Prikaži moguće rješenje i ispis

```
temperatura = 9

if temperatura < 2:
    print("Temperatura je preniska.")
elif temperatura <= 8:
    print("Temperatura je dopuštena.")
else:
    print("Temperatura je previsoka.")

print("Izmjerena temperatura:", temperatura)
```

Temperatura je previsoka.
Izmjerena temperatura: 9

Vrijednosti 2 i 8 pripadaju dopuštenom rasponu. Završni ispis nalazi se izvan grananja pa se izvršava u svim slučajevima.

14.2 Rješenje zadatka 2

▼ Prikaži moguće rješenje i ispis

```
racun_aktivan = True
nepodmirene_obveze = True

if not racun_aktivan:
    print("Korisnički račun nije aktivan.")
elif nepodmirene_obveze:
    print("Prvo podmirite obveze.")
else:
    print("Posudba je moguća.")
```

Prvo podmirite obveze.

Kad račun nije aktivan, prva grana ima prednost neovisno o obvezama. Posudba je moguća samo za aktivan račun bez nepodmirenih obveza. Ugniježđeno grananje također je valjano rješenje.

14.3 Rješenje zadatka 3

▼ Prikaži moguće rješenje i primjer unosa

```
bodovi = int(input("Bodovi: "))

if bodovi < 0 or bodovi > 100:
    print("Bodovi nisu u dopuštenom rasponu.")
elif bodovi < 50:
    print("Potrebno ponavljanje")
elif bodovi < 80:
    print("Zadovoljava")
else:
    print("Vrlo uspješno")
```

Bodovi: 80
Vrlo uspješno

Provjera valjanosti prethodi kategorijama. Provjerite `-1`, `0`, `49`, `50`, `79`, `80`, `100` i `101`. Podrazumijeva se unos cijelog broja, ovaj program ne obrađuje tekst poput `abc`.

14.4 Rješenje zadatka 4

▼ Prikaži moguće rješenje i ispis

```
for broj in range(12, 21):
    print(f"Uzorak {broj}")

print("Označavanje je završeno.")
```

Uzorak 12
Uzorak 13
Uzorak 14
Uzorak 15
Uzorak 16
Uzorak 17
Uzorak 18
Uzorak 19
Uzorak 20
Označavanje je završeno.

Završna granica je `21` jer želimo obraditi i vrijednost `20`.

14.5 Rješenje zadatka 5

▼ Prikaži moguće rješenje i ispis

```
for minuta in range(30, 0, -5):
    print(f"Preostalo minuta: {minuta}")

print("Vrijeme je isteklo")
```

```
Preostalo minuta: 30
Preostalo minuta: 25
Preostalo minuta: 20
Preostalo minuta: 15
Preostalo minuta: 10
Preostalo minuta: 5
Vrijeme je isteklo
```

Korak je negativan jer vrijednosti opadaju. Nula je završna granica i ne ispisuje se kao podsjetnik.

14.6 Rješenje zadatka 6

▼ Prikaži moguće rješenje i primjer unosa

```
ukupno = 0.0
broj_vecih = 0

for redni_broj in range(1, 5):
    trosak = float(input(f"Trošak {redni_broj}: "))
    ukupno += trosak
    if trosak > 20:
        broj_vecih += 1

print(f"Ukupno: {ukupno:.2f} EUR")
print(f"Prosjek: {ukupno / 4:.2f} EUR")
print("Troškova većih od 20 EUR:", broj_vecih)
```

```
Trošak 1: 10
Trošak 2: 20
Trošak 3: 25
Trošak 4: 45
Ukupno: 100.00 EUR
Prosjek: 25.00 EUR
Troškova većih od 20 EUR: 2
```

Trošak od točno 20 eura ne povećava brojač jer se traže strogo veći iznosi. Svi iznosi, uključujući taj, ulaze u ukupan trošak i prosjek.

14.7 Rješenje zadatka 7

▼ Prikaži moguće rješenje i ispis

```
brojac = 0
zbroj = 0

for broj in range(10, 36):
    if broj % 4 == 0:
        print(broj)
        brojac += 1
        zbroj += broj

print("Broj vrijednosti:", brojac)
print("Zbroj:", zbroj)
```

```
12
16
20
24
28
32
Broj vrijednosti: 6
Zbroj: 132
```

Moguće je i izravno prolaziti odgovarajućim rasponom s korakom 4, nakon određivanja prve vrijednosti. Oba postupka trebaju dati isti rezultat.

14.8 Rješenje zadatka 8

▼ Prikaži moguće rješenje i ispis

```
zapis = "A2B40C7"
broj_znamenki = 0
zbroj_znamenki = 0

for znak in zapis:
    if znak in "0123456789":
        broj_znamenki += 1
        zbroj_znamenki += int(znak)

print("Broj znamenki:", broj_znamenki)
print("Zbroj znamenki:", zbroj_znamenki)
```

```
Broj znamenki: 4
Zbroj znamenki: 13
```

Nula povećava broj znamenki, ali ne mijenja njihov zbroj. Pretvorba se izvodi tek nakon provjere znaka, pa se slova ne pokušavaju pretvoriti u broj.

14.9 Rješenje zadatka 9

▼ Prikaži moguće rješenje i ispis

```
preostalo_minuta = 32
trajanje_kutije = 7
broj_kutija = 0

while preostalo_minuta >= trajanje_kutije:
    preostalo_minuta -= trajanje_kutije
    broj_kutija += 1
    print(f"Obrađeno: {broj_kutija}, preostalo minuta: {preostalo_minuta}")

print("Ukupno kutija:", broj_kutija)
print("Neiskorišteno minuta:", preostalo_minuta)
```

```
Obrađeno: 1, preostalo minuta: 25
Obrađeno: 2, preostalo minuta: 18
Obrađeno: 3, preostalo minuta: 11
Obrađeno: 4, preostalo minuta: 4
Ukupno kutija: 4
Neiskorišteno minuta: 4
```

Preostale četiri minute nisu dovoljne za petu kutiju. Provjerite i početno vrijeme manje od 7 te vrijeme točno jednako 7 minuta.

14.10 Rješenje zadatka 10

▼ Prikaži moguće rješenje i primjer unosa

```
velicina = input("Veličina S, M ili L: ").strip().upper()

while velicina != "S" and velicina != "M" and velicina != "L":
    print("Prihvatljive su samo oznake S, M i L.")
    velicina = input("Veličina S, M ili L: ").strip().upper()

print("Odabrana veličina:", velicina)
```

```
Veličina S, M ili L: XL
Prihvatljive su samo oznake S, M i L.
Veličina S, M ili L: m
Odabrana veličina: M
```

Usporedbe su potpune i odvojene. Provjera `velicina in "SML"` sama ne bi bila jednako dobra: prihvatila bi i prazan tekst ili podniz `SM`.

14.11 Rješenje zadatka 11

▼ Prikaži moguće rješenje i primjer unosa

```
broj_prihvacenih = 0
zbroj = 0

while True:
    bodovi = int(input("Bodovi od 0 do 10, -1 za kraj: "))
    if bodovi == -1:
        break
    if bodovi < 0 or bodovi > 10:
        print("Rezultat nije u dopuštenom rasponu.")
        continue

    broj_prihvacenih += 1
    zbroj += bodovi

print("Prihvaćenih rezultata:", broj_prihvacenih)
if broj_prihvacenih > 0:
    print(f"Prosjek: {zbroj / broj_prihvacenih:.2f}")
else:
    print("Prosjek nije dostupan.")
```

```
Bodovi od 0 do 10, -1 za kraj: 8
Bodovi od 0 do 10, -1 za kraj: 12
Rezultat nije u dopuštenom rasponu.
Bodovi od 0 do 10, -1 za kraj: 6
Bodovi od 0 do 10, -1 za kraj: -1
Prihvaćenih rezultata: 2
Prosjek: 7.00
```

Završnu oznaku provjeravamo prije provjere raspona. Tako `-1` završava program, dok se ostali negativni brojevi odbijaju. Za unos samo `-1` dobiva se broj prihvaćenih `0` i poruka da prosjek nije dostupan.

14.12 Rješenje zadatka 12

▼ Prikaži moguće rješenje i ispis

```

zapis = "ab-3-c!de-9"
rezultat = ""

for znak in zapis:
    if znak == "!":
        break
    if znak == "-":
        continue
    rezultat += znak

print("Dobiveni tekst:", rezultat)
print("Broj zadržanih znakova:", len(rezultat))

```

Dobiveni tekst: ab3c
 Broj zadržanih znakova: 4

Ovdje postupno gradimo tekst, pa je početna vrijednost prazan tekst. Bez završne oznake obrađuje se cijeli zapis. Ako je **!** prvi znak, rezultat ostaje prazan i ima duljinu nula.

14.13 Rješenje zadatka 13

▼ Prikaži moguće rješenje i ispis

```

broj_dostupnih = 0

for ormar in "XY":
    for pretinac in range(1, 4):
        oznaka = f"{ormar}{pretinac}"
        if ormar == "Y" and pretinac == 2:
            print(oznaka, "nedostupan")
        else:
            print(oznaka, "dostupan")
            broj_dostupnih += 1

print("Dostupnih pretinaca:", broj_dostupnih)

```

X1 dostupan
 X2 dostupan
 X3 dostupan
 Y1 dostupan
 Y2 nedostupan
 Y3 dostupan
 Dostupnih pretinaca: 5

Samo kombinacija ormara **Y** i pretinca **2** označava nedostupno mjesto. Operator **or** pogrešno bi obuhvatio sve pretince ormara **Y** i oba pretinca s brojem **2**.

14.14 Rješenje zadatka 14

▼ Prikaži moguće rješenje i ispis

```
for broj in range(1, 21):  
    if broj % 2 == 0 and broj % 3 == 0:  
        print("Oboje")  
    elif broj % 2 == 0:  
        print("Dva")  
    elif broj % 3 == 0:  
        print("Tri")  
    else:  
        print(broj)
```

```
1  
Dva  
Tri  
Dva  
5  
Oboje  
7  
Dva  
Tri  
Dva  
11  
Oboje  
13  
Dva  
Tri  
Dva  
17  
Oboje  
19  
Dva
```

Zajednički slučaj provjeravamo prvi. Kada bismo prvo izdvojili sve parne brojeve, vrijednost 6 već bi dobila poruku Dva, pa kasnija zajednička provjera ne bi došla na red.

14.15 Rješenje zadatka 15

▼ Prikaži moguće rješenje i objašnjenje

```
broj = 0
zbroj = 0

while broj < 5:
    broj += 1
    if broj == 3:
        continue
    zbroj += broj

print(zbroj)
```

12

U izvornom programu `continue` za vrijednost `3` preskače povećavanje brojača. Sljedeća iteracija ponovno ima istu vrijednost i ponavlja isti put. U ispravku povećavanje prethodi mogućem preskakanju. Zbog toga su usklađeni i početna vrijednost `0` te uvjet `broj < 5`, kako bi se obradili brojevi od `1` do `5`. Zbroj je `1 + 2 + 4 + 5`, odnosno `12`.